



Рекомендательные системы

Для бакалавриата ПМИ, КНАД, ЭАД и магистратуры МОВС

2024/2025



О себе

Образование:

- Магистратура “Наука о данных” ВШЭ, 2018
- Аспирантура ФКН ВШЭ, 2024

Академический опыт:

- Старший преподаватель ФКН, преподаю с 2019
- Научное руководство и со-руководство ВКР студентов ФКН
- Со-автор 15 статей по графам и RecSys (включая 4 на A-conf)

Индустриальный опыт:

- 2 года в исследованиях в фарме, 6 лет ML в финтехе
- Развиваю ML направление рекомендательных систем и персонализации в AI Центре Т-Банка
- Руководжу отделом разработки RecSys 50+ (MLE, Backend, MLOps & SRE, PA, BI, PM)

Преподавательский состав



Марина Ананьева,
преподаватель
[@ananeyvame](#)



Олег Лашинин,
преподаватель
[@lashinin](#)



Михаил Печатов,
преподаватель
[@pechatov](#)



Денис Красильников,
преподаватель
[@deethereal](#)



Вячеслав Юсупов,
ассистент
[@Sllaavva](#)



Константин Чайников,
ассистент
[@Konstantinator](#)



Яна Журович,
ассистент
[@janinaal](#)



Мария Кокоева,
ассистент
[@amadare14](#)



Дарья Прудникова,
ассистент
[@ambdash](#)

Course objectives

Аннотация	<i>Курс нацелен на формирование у слушателя достаточных знаний, навыков и компетенций для построения рекомендательных систем. Курс является прикладным и охватывает все продвинутые темы, необходимые для успешного применения как в индустрии, так и в академических исследованиях. Слушатели курса научатся постановке и имплементации рекомендательных ML моделей на Python от простой коллаборативной фильтрации до современных нейронных сетей.</i>
Цели освоения курса	<i>1. Знать основные подходы рекомендательных систем и алгоритмов к решению задачи ранжирования и рекомендаций пользователям.</i> <i>2. Уметь правильно определять и формализовывать задачу для построения рекомендательной системы.</i> <i>3. Определять и имплементировать функции ранжирования и метрики качества оценки алгоритмов.</i> <i>4. Разрабатывать модели машинного обучения для рекомендательных систем разной сложности на языке программирования Python.</i> <i>5. Объяснять и интерпретировать предсказания на локальном и глобальном уровне моделей, использовать их для отладки ошибок и аномалий.</i> <i>6. Разрабатывать базовые API сервисы для рекомендательных моделей с помощью Docker.</i>
Пререквизиты курса (необязательно)	<i>1. Линейная алгебра</i> <i>2. Теория вероятностей</i> <i>3. Программирование на Python</i> <i>4. Алгоритмы и структуры данных</i> <i>5. Машинное обучение</i> <i>6. Глубинное обучение</i>

Course outline

Неделя	Тема	Подробнее:
1	Введение в рекомендательные системы	Обзор прикладных задач. Формализация задач ранжирования и рекомендаций. Типы (implicit, explicit), форматы (e.g. матрица взаимодействий) данных. Функции и метрики качества ранжирования. Классификация рекомендательных моделей.
2	Базовые подходы	Неперсональные рекомендации и бенчмарки на статистиках. Подходы на основе user-based и item-based метрик близости. Линейные модели на примере EASE, SLIM.
3	Матричная факторизация	Матричные разложения на примере SVD. Проблемы разреженности и высокой размерности данных.
4	Коллаборативная фильтрация	Имплементация implicit ALS. Техники препроцессинга матрицы взаимодействий. Нейросетевая коллаборативная фильтрация Neural CF.
5	Контентные подходы	Способы добавления user и item признаков в модели. Разбор моделей LightFM и DSSM.
6	Гибридные модели	Таксономия. Архитектура двухуровневой модели. Особенности разбиения датасета, кросс-валидации, подбора параметров пайплайна. Градиентный бустинг как end-to-end рекомендательная модель и как ранкер второго уровня. Нейросетевой ранжирование как ранкер.

Course **outline**

Неделя	Тема	Подробнее:
7	Нейросетевые модели для задачи next-item рекомендаций	Постановка задачи рекомендаций на последовательных данных. GRU4Rec, SASRec, BERT4Rec.
8	Контекстные рекомендации	Time-aware, time-dependent рекомендательные подходы. Temporal reasoning. Техники учета времени в моделях.
9	Нейросетевые модели для задачи NBR (next-basket recommendation)	Задача предсказания следующей корзины. Алгоритмы TIFU KNN, TAIW и другие алгоритмы.
10	Автоэнкодеры и вариационные автоэнкодеры	Особенности использования автоэнкодеров и вариационных автоэнкодеров для рекомендаций. Имплементация Rec-VAE, Multi-VAE, Multi-DAE.
12	Интерпретируемость и объяснимость моделей	Model-intrinsic и model-agnostic подходы. Векторы Шепли. Обзор нейросетевых подходов для рекомендаций с объяснениями
13	Uplift и экономические рекомендательные системы	Разбор прикладных метрик и подходов для задачи получения uplift от показа рекомендаций вместо задачи предугадывания действий.

Course outline

Неделя	Тема	Подробнее:
14	Multi-task и cross-domain рекомендательные модели	Attribute, type и item уровни обучения. Content-based, embedding-based, rating pattern based transfer подходы. Single, dual и multi-task таргеты в CDR моделях.
15	RL для рекомендательных систем	Offline и online RL-based подходы для рекомендательных систем. TD3, TD3+BC.
16	Контентные рекомендации на мультимодальных данных	Доменные transformed-based модели на основе текстов, изображений и видео.
17	A/B тестирование в RecSys	Онлайн и оффлайн метрики, proxy метрики. Вспомогательные метрики (novelty, diversity, serendipity, coverage). Дизайн A/B тестов. Многорукие бандиты
18	Large scale RecSys	Особенности индустриальной разработки. RecSys ML System Design. Типовые архитектуры систем и моделей для высоконагруженных систем.
19	Сервис рекомендательной системы	Разбор и имплементация простой архитектуры сервиса рекомендательной системы. Использование Docker, FAST API, Inference server (Triton), Redis для поднятия сервиса. Мониторинг стабильности и деградации модели.
20	Прикладные аспекты и тренды в рекомендательных системах	Дополнительные техники и подходы, используемых на практике. Индустриальные и академические тренды в рекомендательных системах. LLM в RecSys

Оценивание

Final Grade = 0.3 * Home Assignments + 0.3 * Weekly Quizzes + 0.4 * Exam

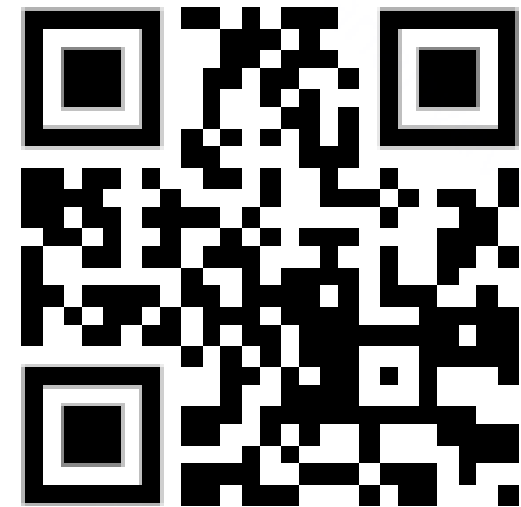
где:

- **Home Assignments** – 5 домашних работ, из них 4 в Jupyter Notebook и 1 на платформе Kaggle (max 10 баллов за каждую).
- **Weekly Quizzes** - 19 квизов по материалам семинаров и лекций, сдаются перед началом каждого следующего занятия в Google Forms (арифметическое среднее за все квизы, max 10 баллов за каждый).
- **Exam** – устный экзамен в формате решения case-study построения рекомендательной системы для бизнеса (max 10 баллов).

Полезные ссылки

Чат курса в Telegram

[ссылка](#)



Wiki ФКН с материалами

[ссылка](#)



Github репозиторий

[ссылка](#)



Таблица с оценками

[ссылка](#)





Введение в контекст

Рекомендательные системы и персонализация

Рекомендательные системы - это подкласс информационных систем (Information Filtering),
цель которых выдавать наиболее релевантные объекты.

A **recommender system (RecSys)**, or a **recommendation system** (sometimes replacing *system* with terms such as *platform*, *engine*, or *algorithm*), is a subclass of [information filtering system](#) that provides suggestions for items that are most pertinent to a particular user. *

* Ricci, Francesco; Rokach, Lior; Shapira, Bracha (2022). "Recommender Systems: Techniques, Applications, and Challenges". In Ricci, Francesco; Rokach, Lior; Shapira, Bracha (eds.). *Recommender Systems Handbook* (3 ed.). New York: Springer. pp. 1–35

Roy, Deepjyoti; Dutta, Mala (2022). "A systematic review and research perspective on recommender systems". *Journal of Big Data*. 9 (59)

Рекомендательные системы и персонализация

Рекомендательные системы - это подкласс информационных систем (Information Filtering),
цель которых выдавать наиболее релевантные объекты.

Может рассматриваться как:

- *ML и не-ML задача

- *С персонализацией и без персонализации

Рекомендательные системы и персонализация

Рекомендательные системы - это подкласс информационных систем (Information Filtering),
цель которых выдавать наиболее релевантные объекты.

Может рассматриваться как:

- *ML и не-ML задача
- *С персонализацией и без персонализации

Персонализация - это набор различных способов учета интересов и потребностей пользователей на основе имеющихся о них данных.

Рекомендательные системы и персонализация

* С персонализацией и без персонализации

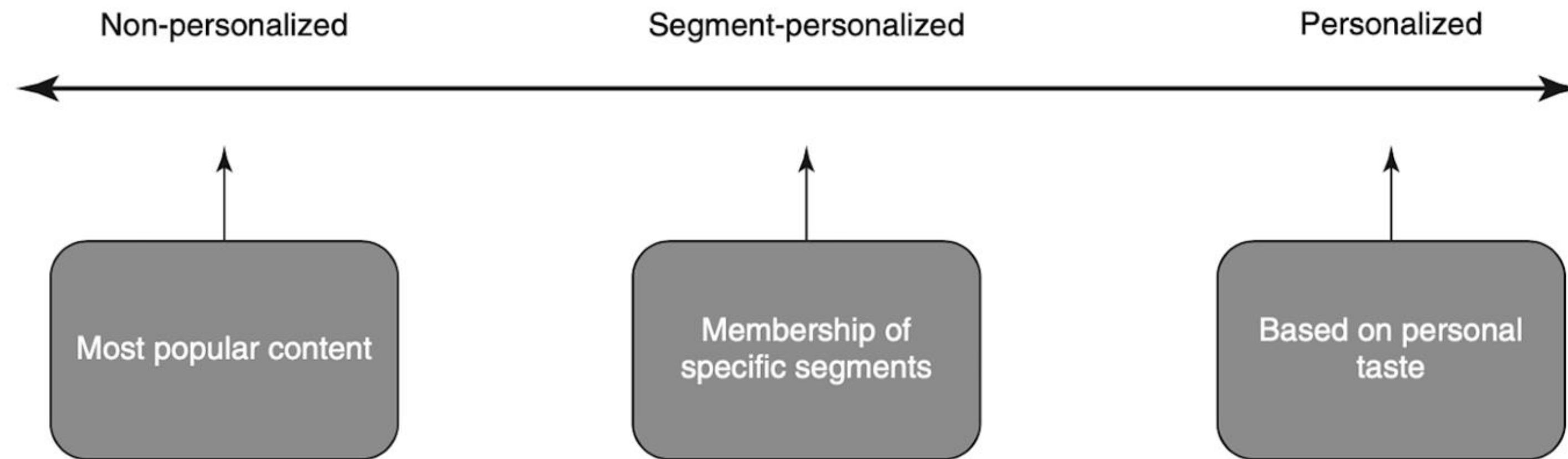
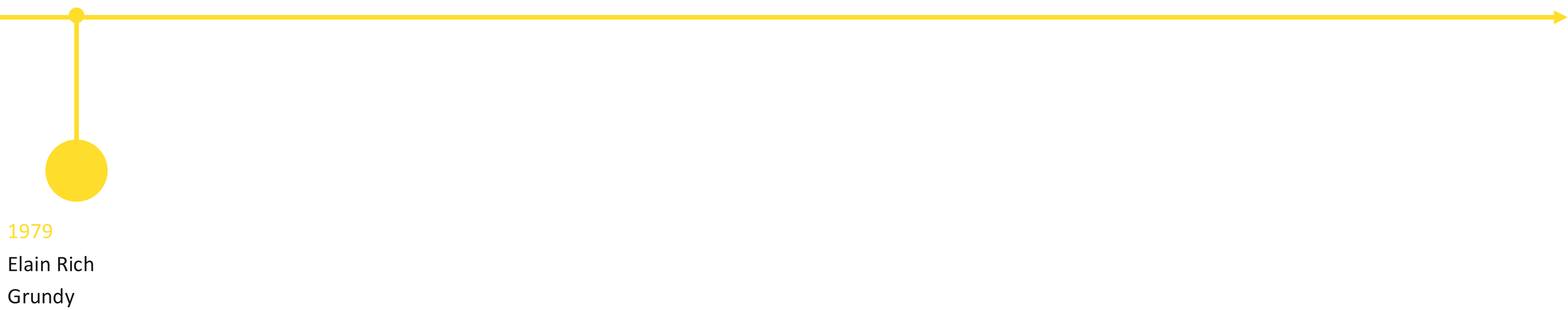


Figure 14.2 Grades of personalization

Краткая история развития RecSys



Краткая история развития RecSys

1992

Белкин и Крофт,
сравнение IR & IF



1979

Elain Rich
Grundy

Краткая история развития RecSys

1992

Белкин и Крофт,
сравнение IR & IF



Краткая история развития RecSys

1992

Белкин и Крофт,
сравнение IR & IF

1994

GroupLens
сервис рекомендации новостей



Краткая история развития RecSys

1992

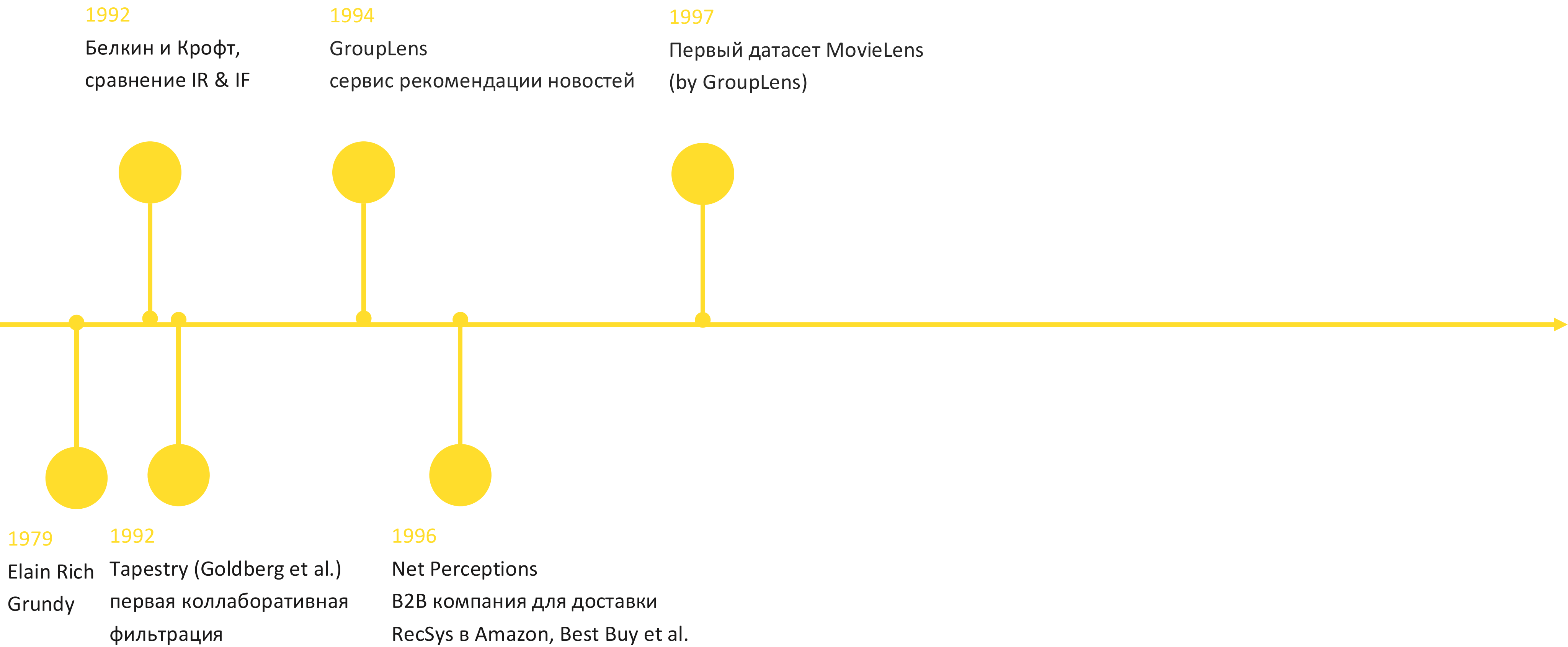
Белкин и Крофт,
сравнение IR & IF

1994

GroupLens
сервис рекомендации новостей



Краткая история развития RecSys



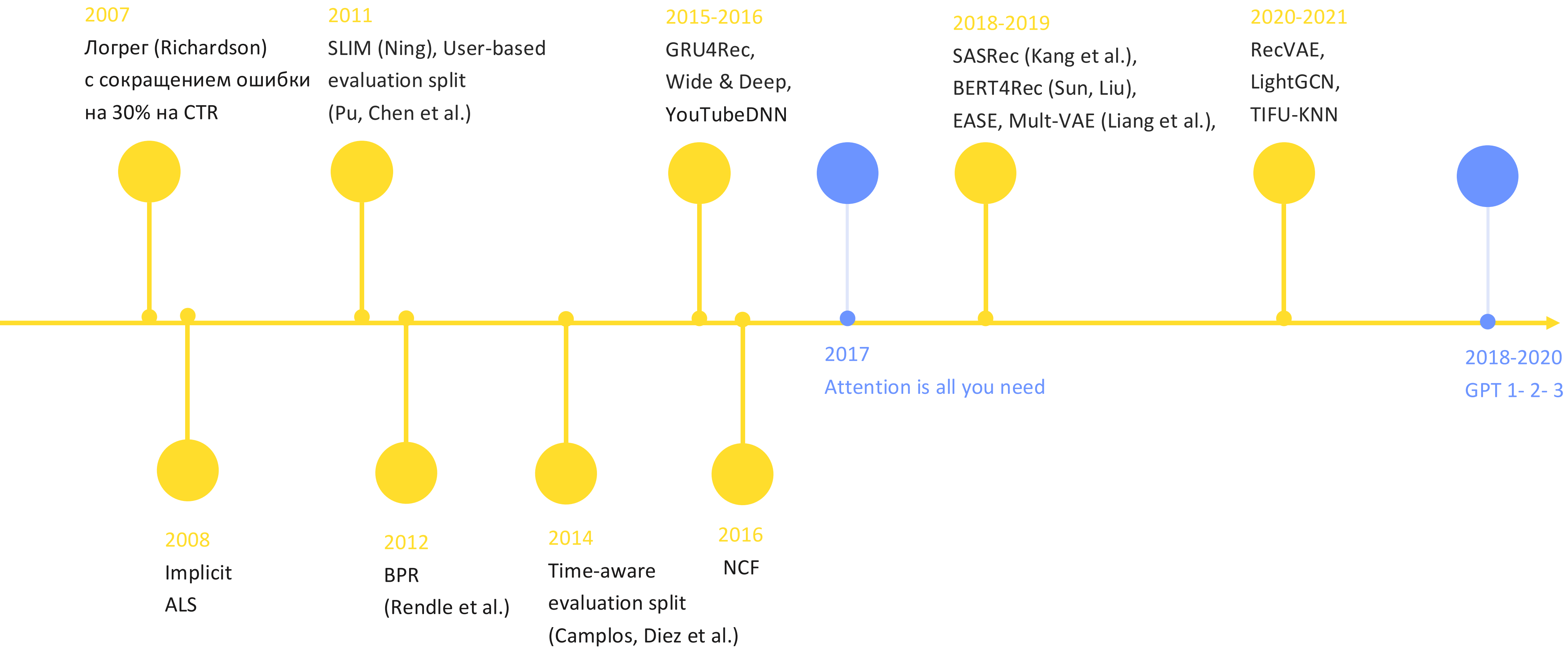
Краткая история развития RecSys



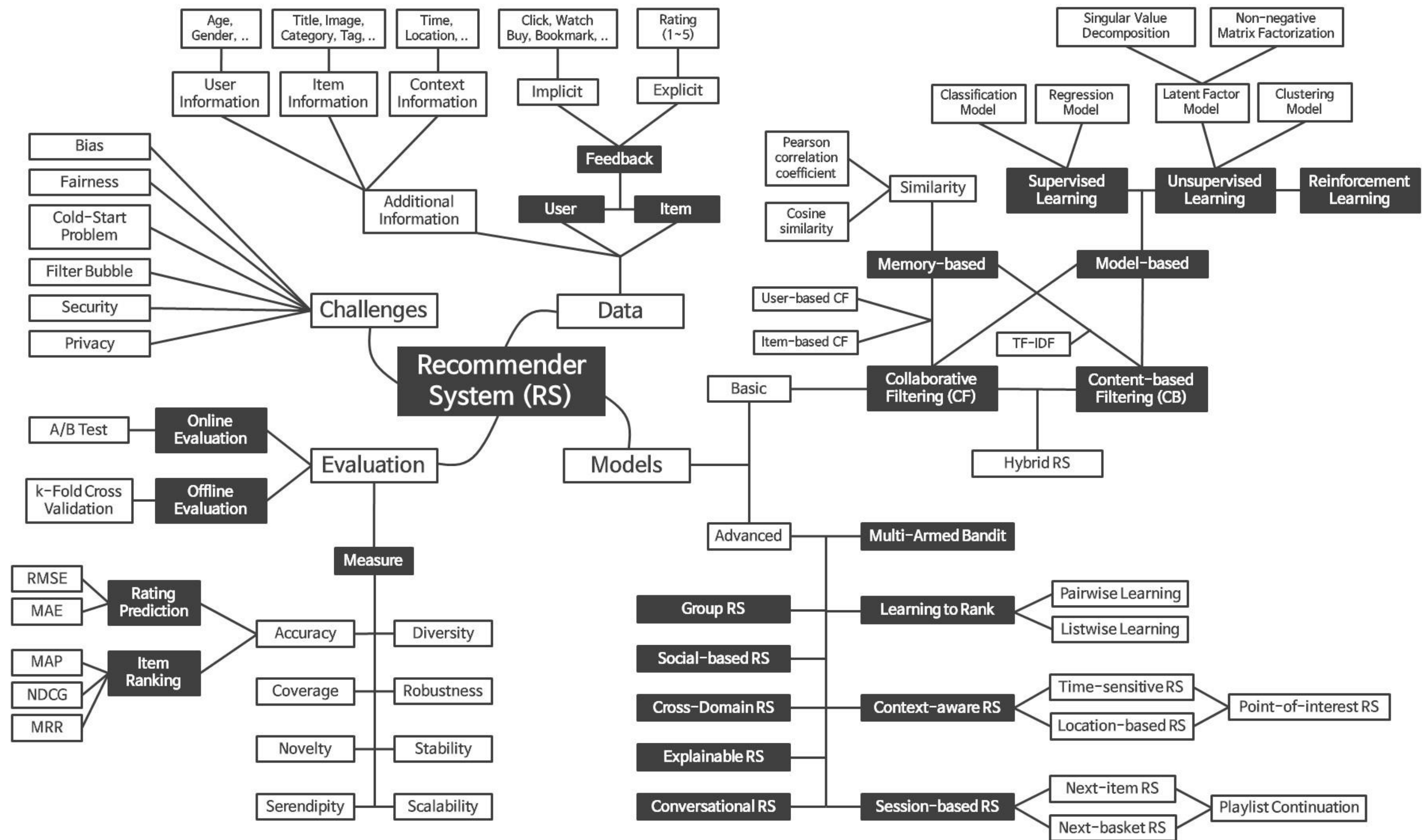
Краткая история развития RecSys



Краткая история развития RecSys



Таксономия



Основные домены в RecSys

- **E-commerce** (Amazon, Ozon, Wildberries, др.)
- **Социальные сети** (TikTok и др.)
- **Стриминговые и контентные платформы** (Netflix, YouTube, др.)
- **Музыка** (Spotify, Яндекс.Музыка и др.)
- **Поиск** (Google, Perplexity, Яндекс и др.)
- **Новостные и медиа издания** (Дзен, ТЖ и др)
- **Финтех и банкинг**
- **Медицина**
- **AI-based продукты**

и многое другое!

Зачем нужны рекомендации пользователям?

Зачем рекомендации **пользователю?**



Более релевантные предложения, удовлетворяющие потребности, интересы и ожидания

Зачем рекомендации **пользователю?**



Более релевантные предложения, удовлетворяющие потребности, интересы и ожидания



Снижение негатива (от лишних коммуникаций, нерелевантных предложений)

Зачем рекомендации **пользователю?**



Более релевантные предложения, удовлетворяющие потребности, интересы и ожидания



Снижение негатива (от лишних коммуникаций, нерелевантных предложений)



Экономия времени и сил на поиск

Зачем рекомендации **пользователю?**



Более релевантные предложения, удовлетворяющие потребности, интересы и ожидания



Снижение негатива (от лишних коммуникаций, нерелевантных предложений)



Экономия времени и сил на поиск



Положительный клиентский опыт от удобства использования сервиса

Зачем рекомендации **пользователю?**



Более релевантные предложения, удовлетворяющие потребности, интересы и ожидания



Снижение негатива (от лишних коммуникаций, нерелевантных предложений)



Экономия времени и сил на поиск



Положительный клиентский опыт от удобства использования сервиса

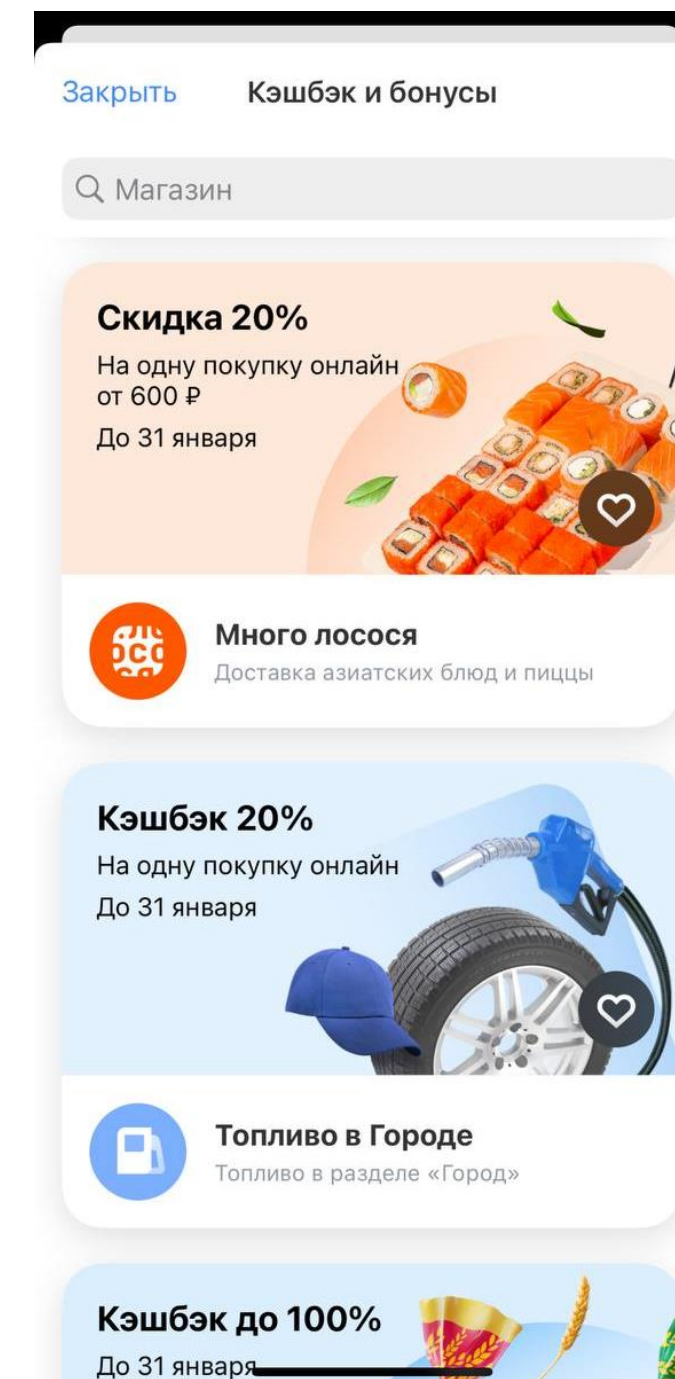
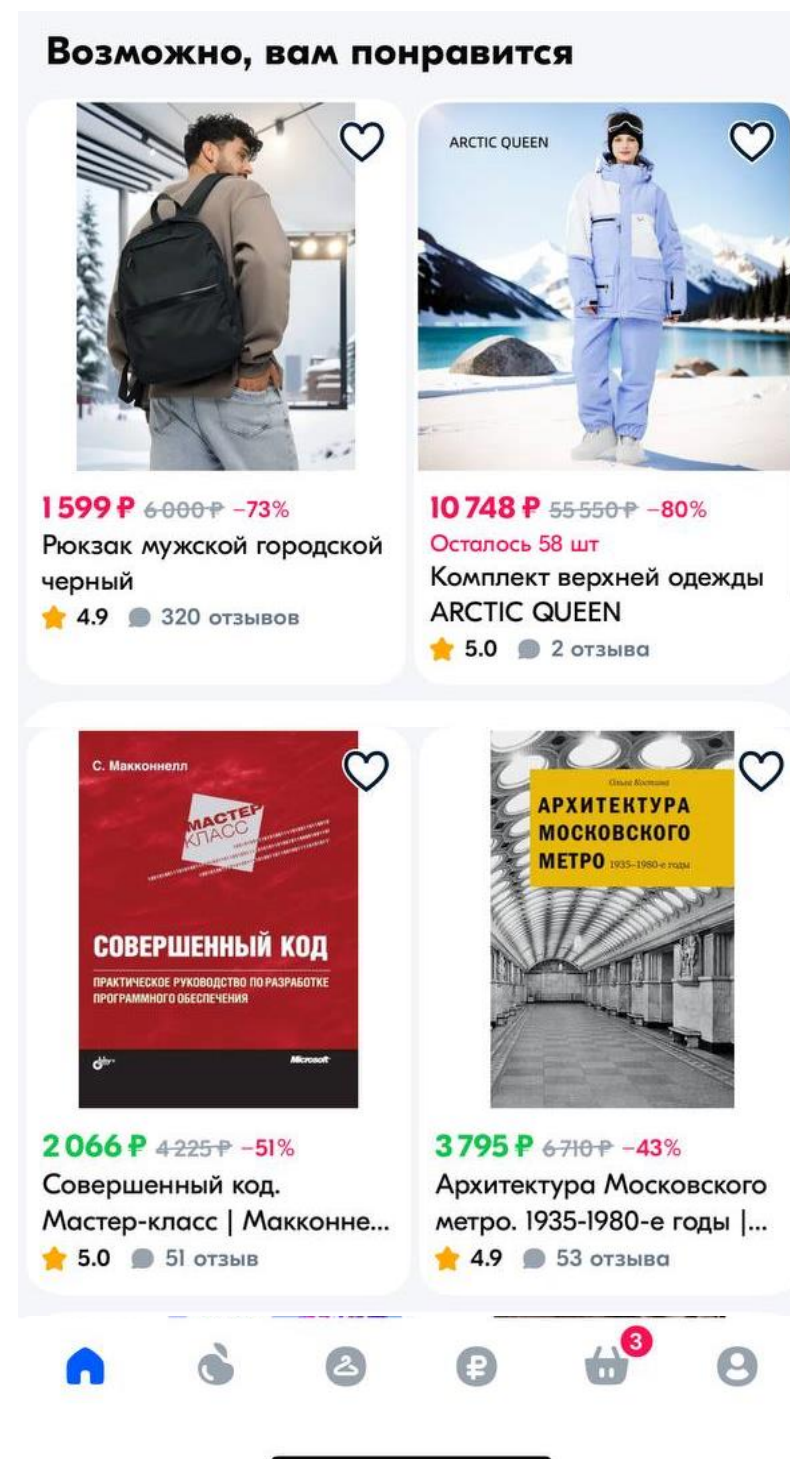


Exploration при экономии сил и времени на поиск

Зачем рекомендации **пользователю?**

Более релевантный контент, акции, продукты, товары, магазины, коммуникации.

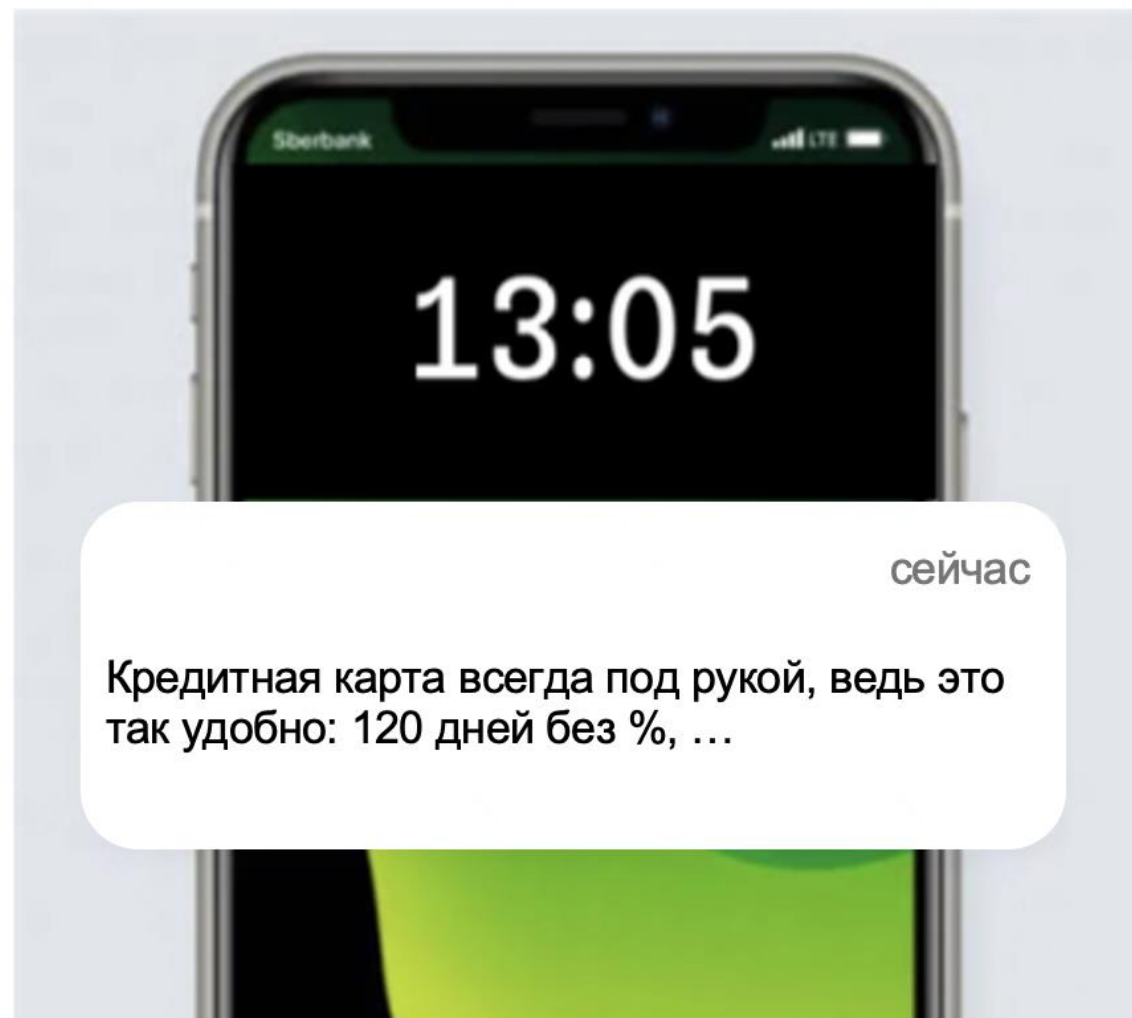
Примеры: E-commerce рекомендации товаров, магазинов, книг и других товаров



Зачем рекомендации **пользователю?**

Снижение негатива (удержания лишних коммуникаций, нерелевантных предложений)

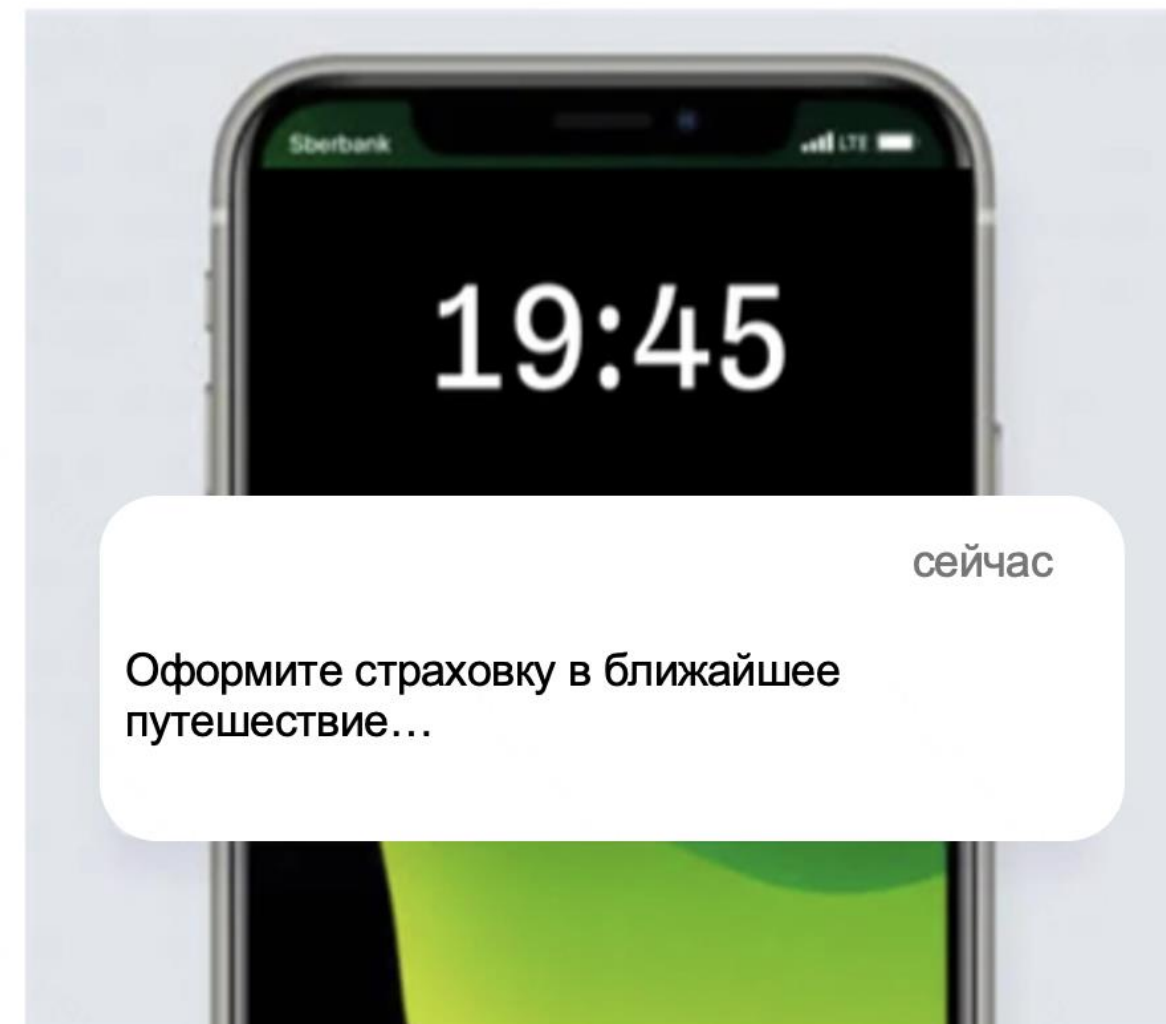
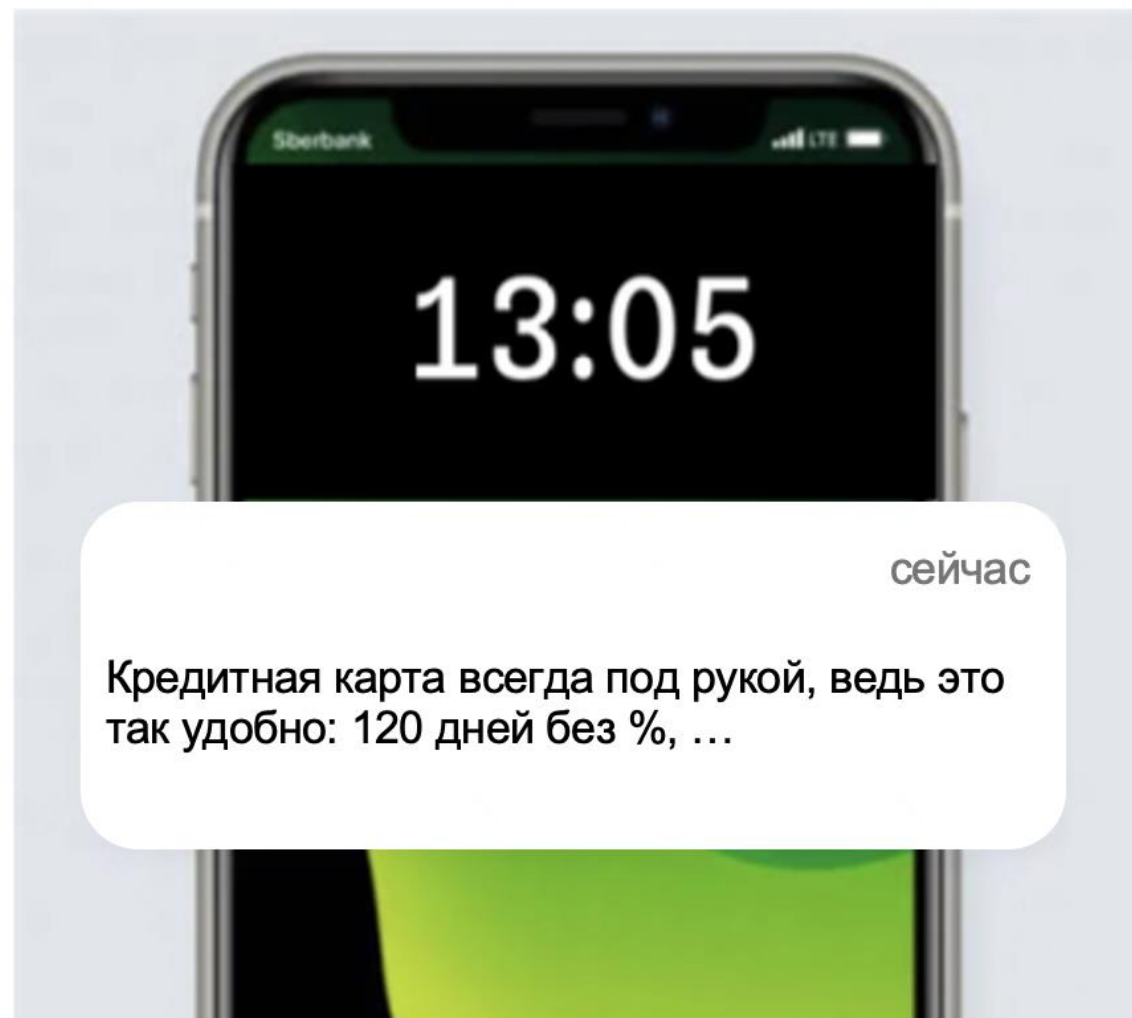
Примеры: отбор целевой аудитории для маркетинговой акции, прогнозирование времени отправки пуша



Зачем рекомендации **пользователю?**

Снижение негатива (удержания лишних коммуникаций, нерелевантных предложений)

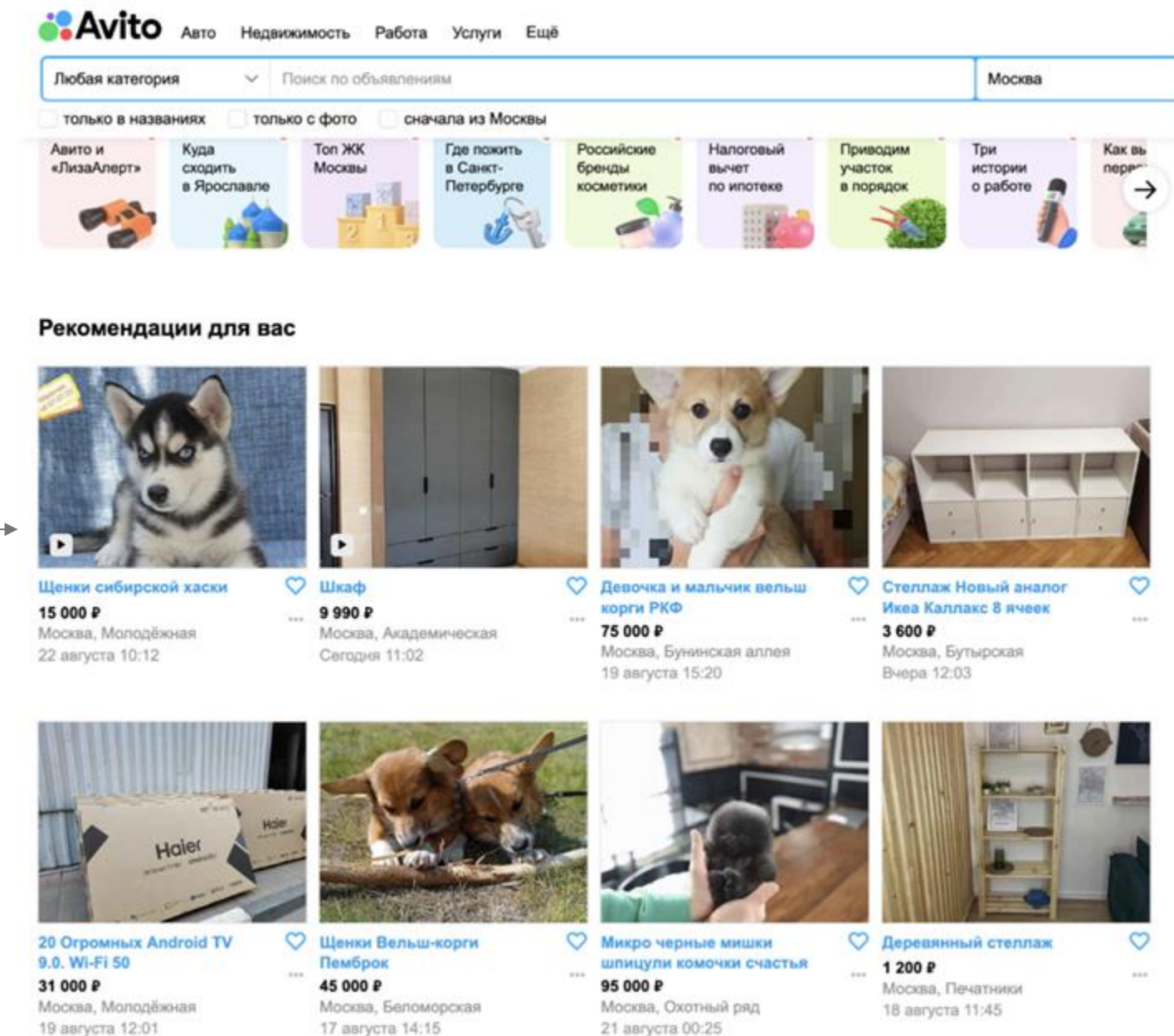
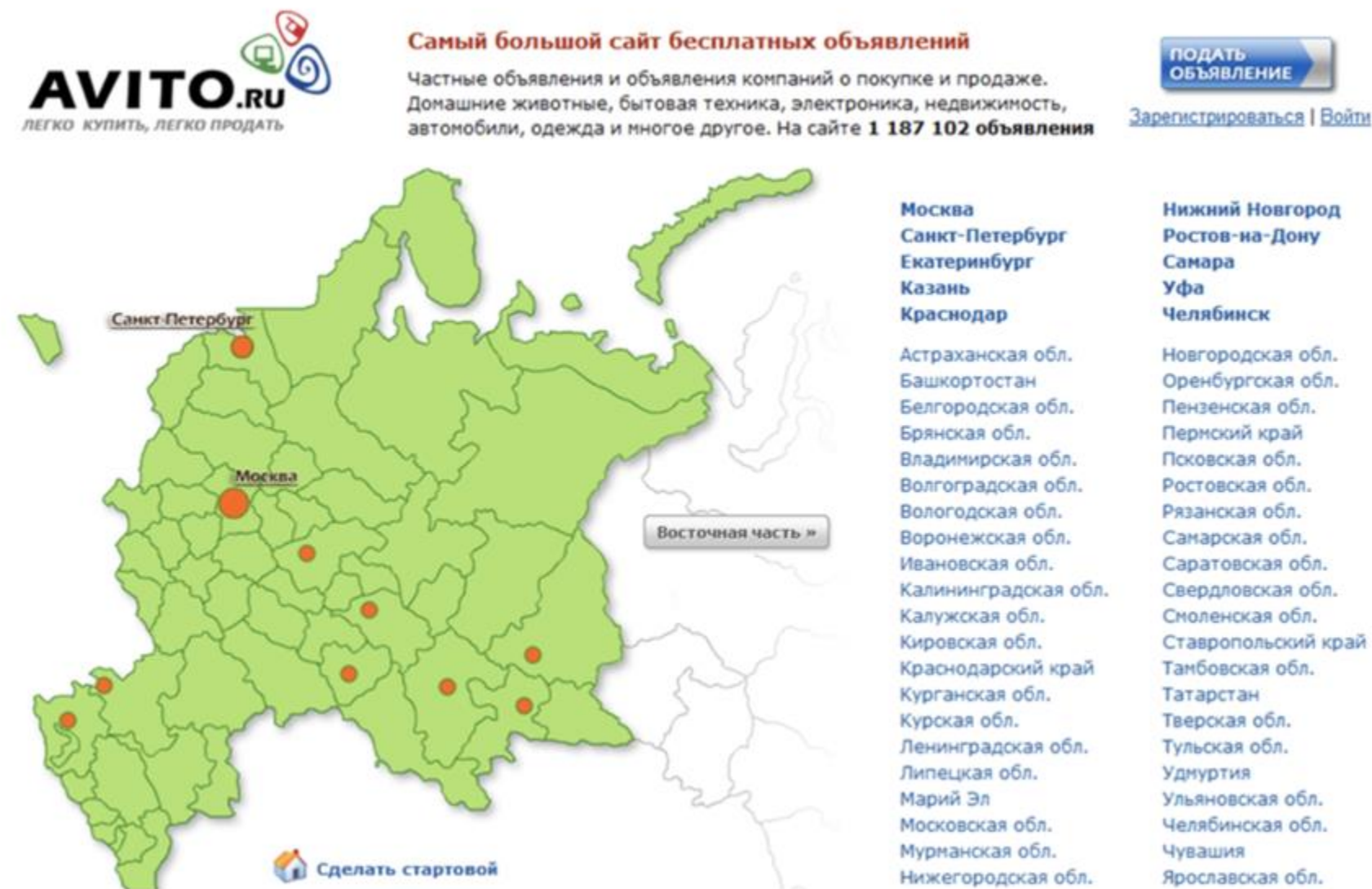
Примеры: отбор целевой аудитории для маркетинговой акции, прогнозирование времени отправки пуша



Зачем рекомендации пользователю?

Положительный клиентский опыт от сервиса

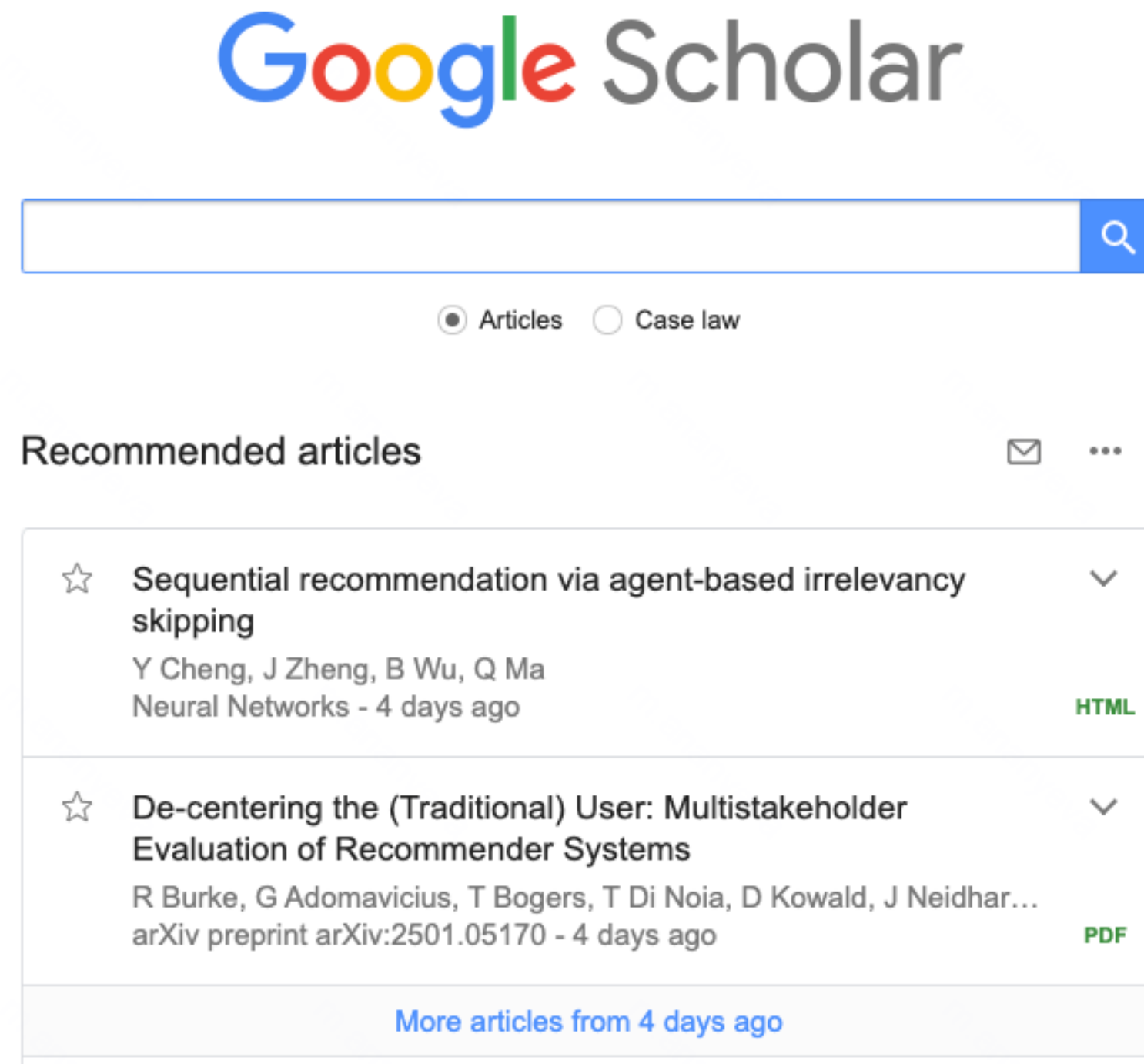
Примеры: персональная лента объявлений на Avito на основе предыдущих запросов вместо гео-карты



Зачем рекомендации **пользователю?**

Экономия времени и сил на поиск внутри сервиса

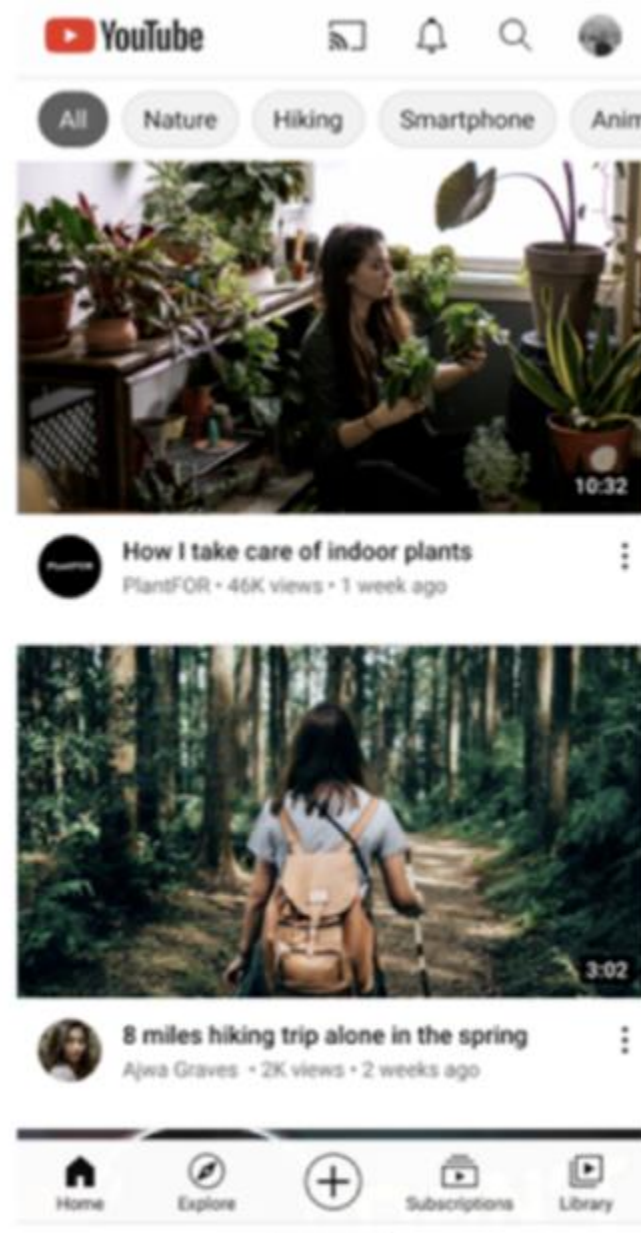
Примеры: рекомендации научных статей и подписок на авторов в Google Scholar



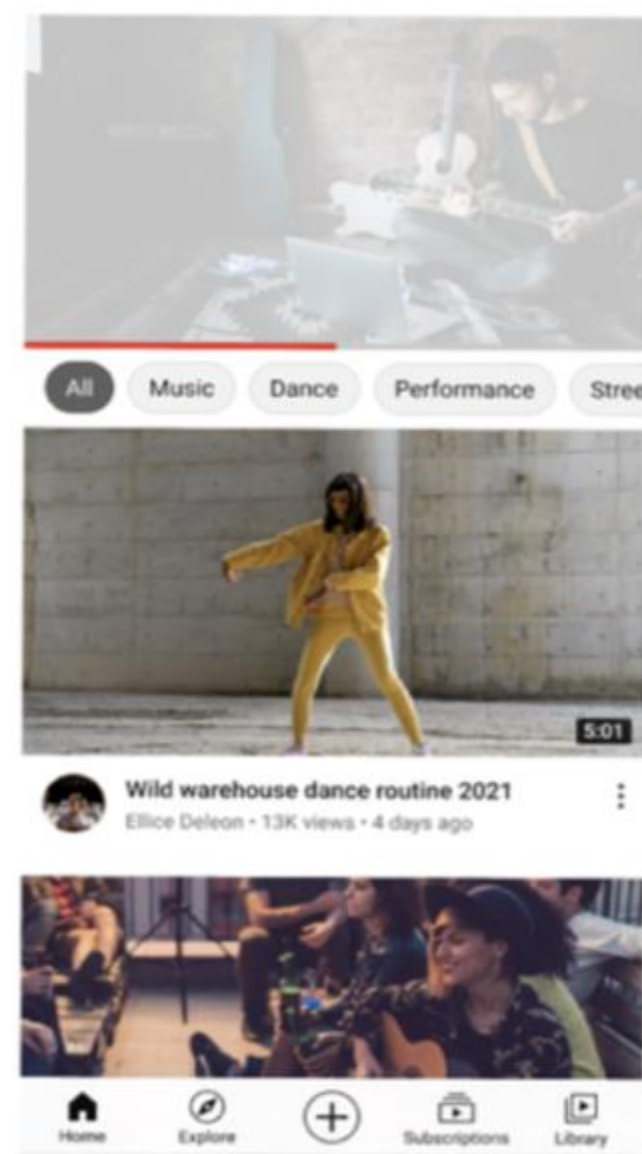
Зачем рекомендации **пользователю?**

Exploration

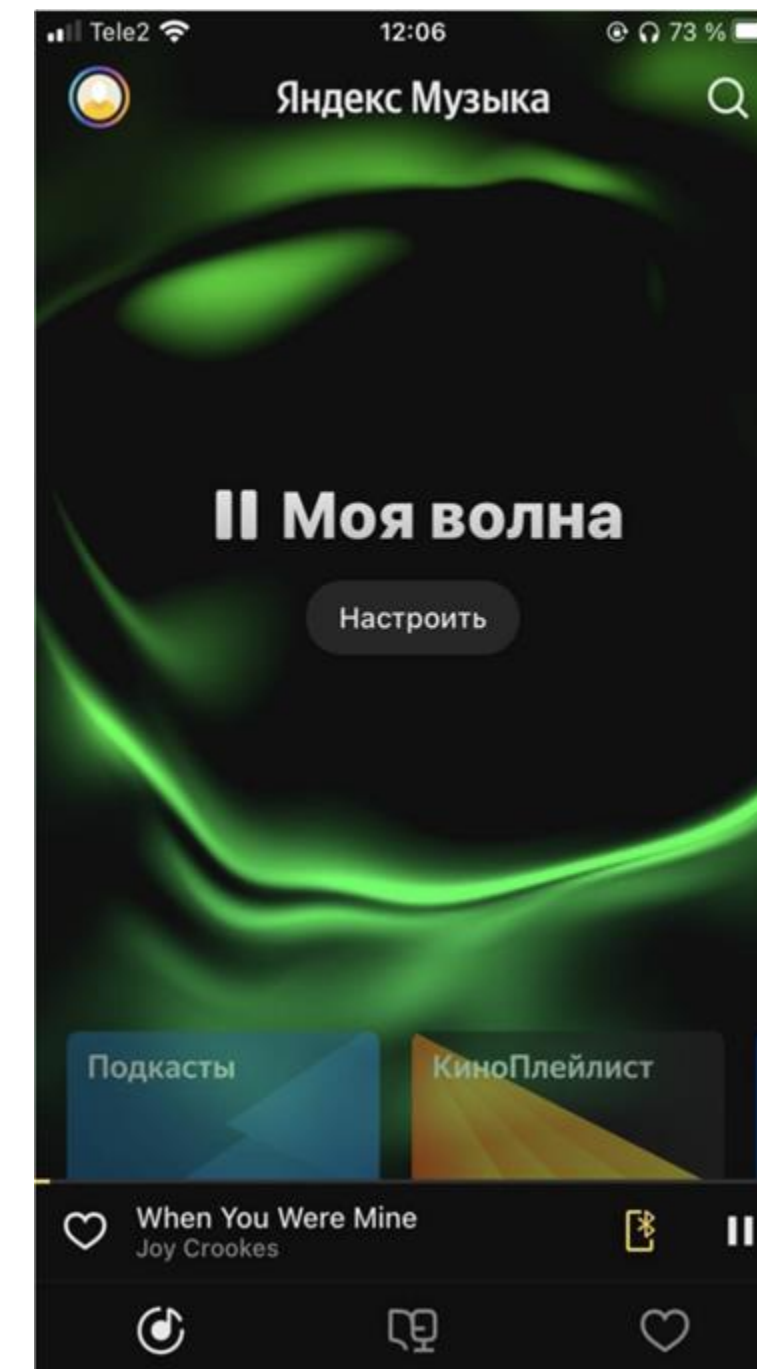
Примеры: рекомендации видео на YouTube, TikTok, рекомендации музыки в Spotify и Яндекс.Музыке.



Recommendations on homepage.



Recommendations on "Up next."



Зачем нужны рекомендации компаниям?

Зачем рекомендации компаниям?



Увеличение продуктовых и бизнес-метрик (конверсии в просмотры, клики, покупки)

Зачем рекомендации компаниям?



Увеличение продуктовых и бизнес-метрик (конверсии в просмотры, клики, покупки)



Дополнительная монетизация (прирост к доходу или экономии)

Зачем рекомендации компаниям?



Увеличение продуктовых и бизнес-метрик (конверсии в просмотры, клики, покупки)



Дополнительная монетизация (прирост к доходу или экономии)



Вовлеченность пользователей (e.g. MAU, retention, timespent)

Зачем рекомендации компаниям?



Увеличение продуктовых и бизнес-метрик (конверсии в просмотры, клики, покупки)



Дополнительная монетизация (прирост к доходу или экономии)



Вовлеченность пользователей (e.g. MAU, retention, timespent)



Лояльность пользователей (e.g. NPS)

Зачем рекомендации компаниям?



Увеличение продуктовых и бизнес-метрик (конверси в просмотры, клики, покупки)



Дополнительная монетизация (прирост к доходу или экономии)



Вовлеченность пользователей (e.g. MAU, retention, timespent)



Лояльность пользователей (e.g. NPS)



Улучшение клиентского опыта (e.g. CSAT)

RecSys в числах индустриальных компаний

80% качественных просмотров контента Netflix*

*The Netflix Recommender System: Algorithms, Business Value, and Innovation”, Netflix Recommendation Engine Drives 75% of Viewership, Dylan Love, Business Insider. Apr 9, 2012

60% кликов в просмотр с главной YouTube*

*The YouTube video recommendation system. In Proceedings of the fourth ACM conference on Recommender systems. 293–296

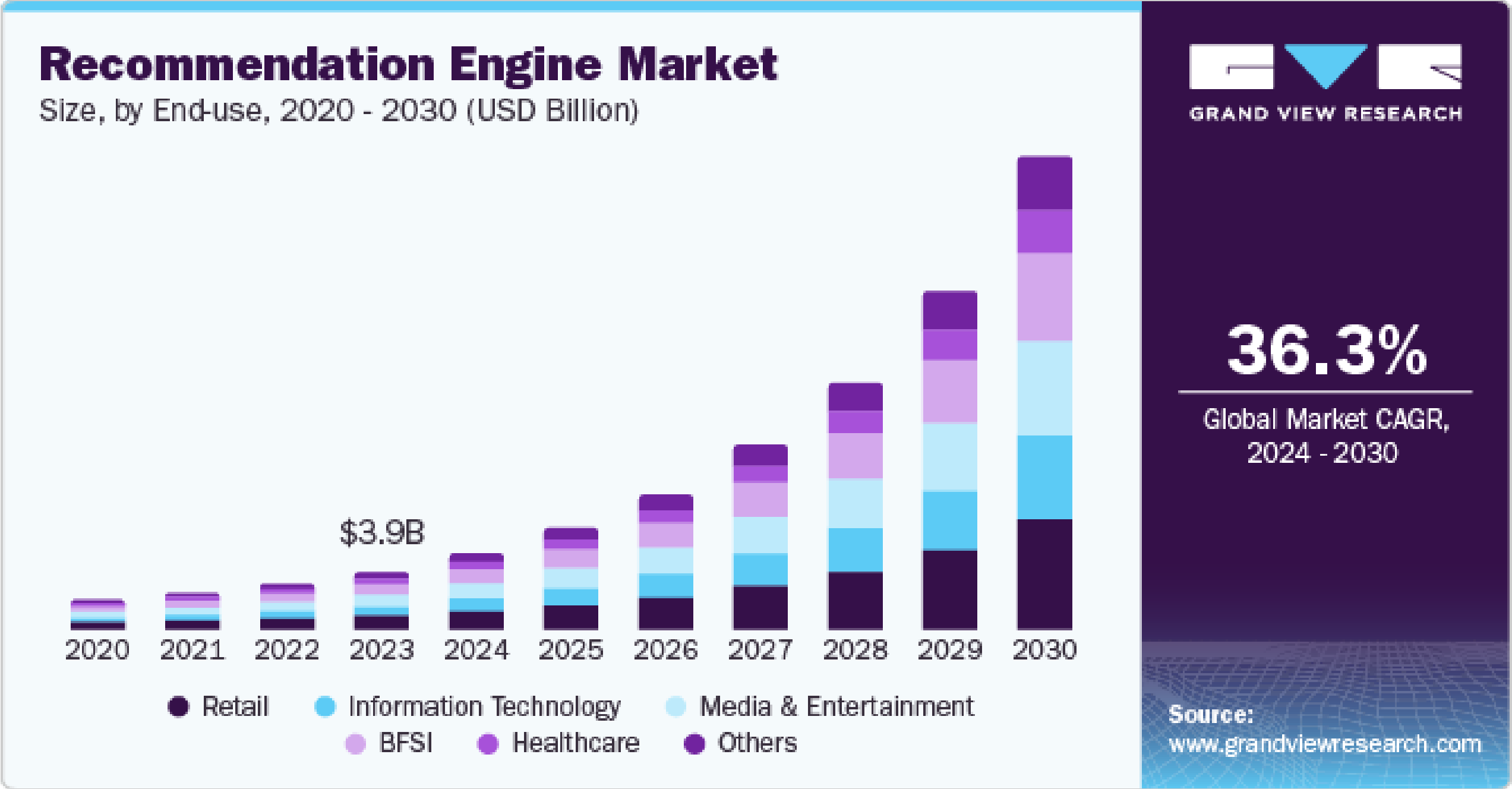
35% выручки Amazon по отчету McKinsey*

* <https://www.mckinsey.com/industries/retail/our-insights/how-retailers-can-keep-up-with-consumers>

40% прослушиваний Spotify

RecSys Market Cup

> 5B \$ в 2024 году
~ 35B \$ к 2029 году



According to Grand View Research, Recommendation Systems Market Cup will be 39.4B dollars in 2029.



Формализация задачи

Постановка задачи ранжирования

X — множество объектов.

$X^l = \{x_1, \dots, x_l\}$ — обучающая выборка

$i < j$ на парах $(i, j) \in \{1, \dots, l\}^2$

Задача:

Построить ранжирующую функцию $a : X \rightarrow \mathbb{R}$ такую, что: $i < j \Rightarrow a(x_i) < a(x_j)$.

Линейная модель ранжирования:

$a(x; w) = \langle x, w \rangle$, где $x \mapsto (f_1(x), \dots, f_n(x)) \in \mathbb{R}^n$ — вектор признаков объекта x .

Пример 1. Ранжирование поисковой выдачи

D — коллекция текстовых документов.

Q — множество запросов.

$D_q \subseteq D$ — множество документов, найденных по запросу q .

$X = Q \times D$ — объектами являются пары (запрос, документ): $x \equiv (q, d), q \in Q, d \in D_q$.

Y — упорядоченное множество рейтингов.

$y : X \rightarrow Y$ — оценки релевантности, поставленные ассессорами (экспертами): чем выше оценка $y(q, d)$, тем релевантнее документ d по запросу q .

Правильный порядок определен только между документами, найденными по одному и тому же запросу q : $(q, d) < (q, d') \Leftrightarrow y(q, d) < y(q, d')$.

Релевантные ответы запросу q — это список документов d , упорядоченных с помощью функции ранжирования $a(q, d)$.

Пример 2. Рекомендации пользователям

U — пользователи.

I — предметы (фильмы, книги и т.д.).

$X = U \times I$ — объектами являются пары (пользователь, предмет).

$$(u, i) \prec (u, i') \Leftrightarrow y(u, i) < y(u, i')$$

Рекомендация пользователю u — это список предметов i , упорядоченный с помощью функции ранжирования $a(u, i)$.

Пример 2. Рекомендации пользователям

$$R = \{(u, i, r_{iu})\}$$

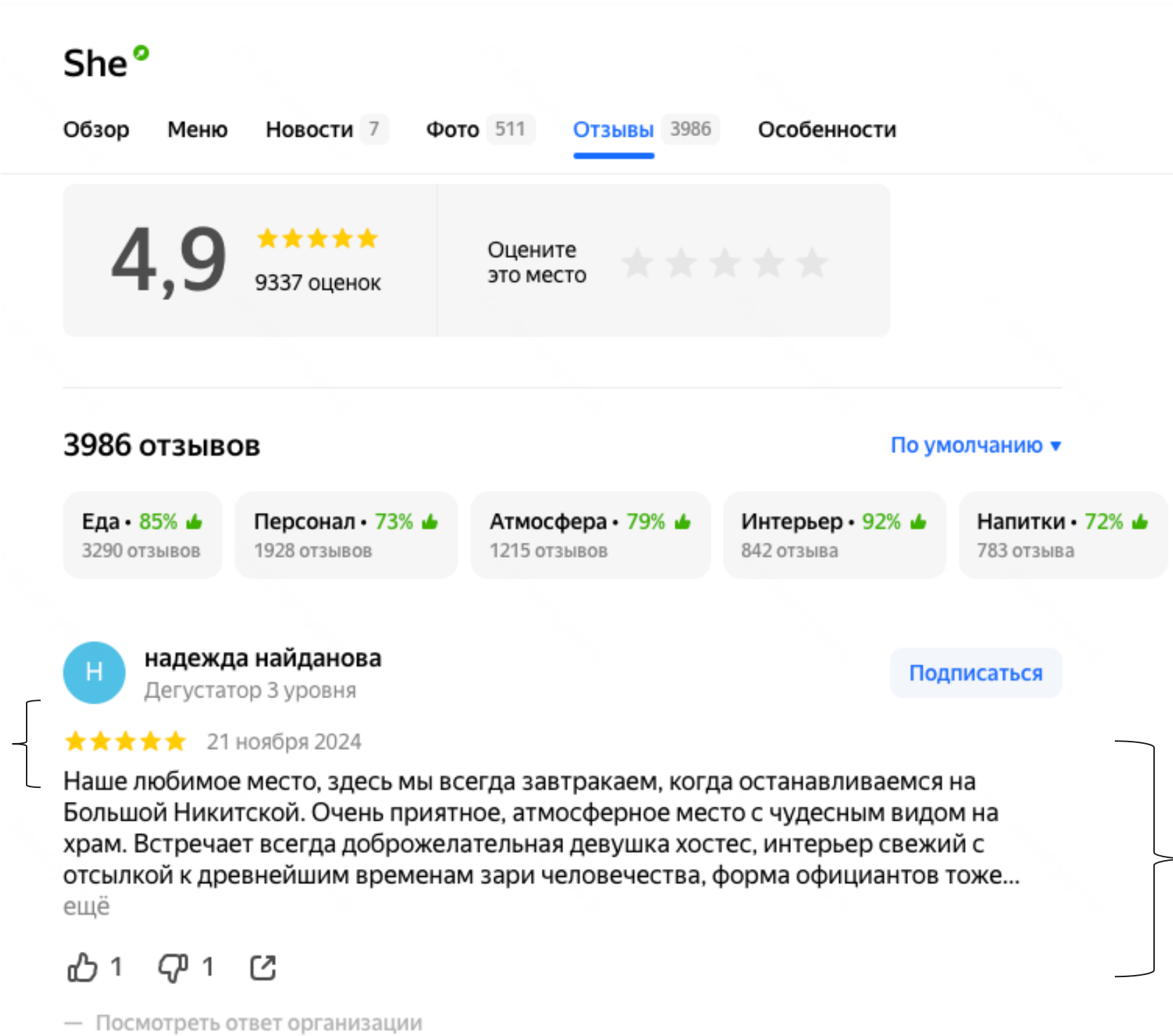




Типы данных

Explicit & implicit данные

Explicit feedback / Явные данные - тип данных, когда известны оценки пользователей в результате взаимодействия с объектом. Например, датасет с оценками пользователей насколько им понравились посещенные рестораны.



5-балльная оценка

Отзыв пользователя

Explicit & implicit данные

Explicit feedback / Явные данные - тип данных, когда известны оценки пользователей в результате взаимодействия с объектом. Например, датасет с оценками пользователей насколько им понравились посещенные рестораны.

Implicit feedback / Неявные данные - тип данных, когда нам неизвестно насколько понравился или не понравился объект после того, как пользователь провзаимодействовал с ним.

User ID	Item ID	Timestamp (view start)	Timestamp (view end)	Clicks
45480	585	1998-03-30 19:08:35	1998-03-30 19:09:34	1
66757	709	1997-11-18 10:33:05	1997-11-18 10:35:25	0
26524	255	1997-12-27 10:00:45	1997-12-27 10:03:26	2
17241	144	1998-02-22 00:01:54	1998-02-22 00:02:48	0
77144	896	1998-02-11 22:10:27	1998-02-11 22:16:27	0
24073	474	1998-02-19 22:06:17	1998-02-19 22:08:09	0

Explicit & implicit данные

Explicit feedback / Явные данные - тип данных, когда известны оценки пользователей в результате взаимодействия с объектом. Например, датасет с оценками пользователей насколько им понравились посещенные рестораны.

Implicit feedback / Неявные данные - тип данных, когда нам неизвестно насколько понравился или не понравился объект после того, как пользователь провзаимодействовал с ним.

Матрица взаимодействий - это матрица, у которой по одной оси - id пользователей, по второй оси - id объектов, а на пересечении - оценка/наличие взаимодействия.

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	0	3	0	3	0
User 2	4	0	0	2	0
User 3	0	0	3	0	0
User 4	3	0	4	0	3
User 5	4	3	0	4	0

Explicit & implicit данные

Explicit feedback / Явные данные - тип данных, когда известны оценки пользователей в результате взаимодействия с объектом. Например, датасет с оценками пользователей насколько им понравились посещенные рестораны.

Implicit feedback / Неявные данные - тип данных, когда нам неизвестно насколько понравился или не понравился объект после того, как пользователь провзаимодействовал с ним.

Матрица взаимодействий - это матрица, у которой по одной оси - id пользователей, по второй оси - id объектов, а на пересечении - оценка/наличие взаимодействия.

	item #1		item #N
user #1	0	1	...
....	1	0	...
user #M	...	...	1

implicit ?

Explicit & implicit данные

Explicit feedback / Явные данные - тип данных, когда известны оценки пользователей в результате взаимодействия с объектом. Например, датасет с оценками пользователей насколько им понравились посещенные рестораны.

Implicit feedback / Неявные данные - тип данных, когда нам неизвестно насколько понравился или не понравился объект после того, как пользователь провзаимодействовал с ним.

Матрица взаимодействий - это матрица, у которой по одной оси - id пользователей, по второй оси - id объектов, а на пересечении - оценка/наличие взаимодействия.

	item #1		item #N
user #1	0	1	...
....	1	0	...
user #M	...	...	1

implicit ?

	item #1		item #N
user #1	0	3	...
....	5	0	...
user #M	...	...	1

explicit ?

Explicit или implicit ?

```
'''
```

Пример 1. По 5 клиентам магазина известна история покупок из 10 уникальных товаров.

1 – товар был куплен, 0 – не был.

```
'''
```

```
import numpy as np  
from numpy import random
```

```
random.randint(low=0, high=2, size=(5,10))
```

```
array([[0, 0, 0, 0, 0, 0, 1, 1, 0, 0],  
       [0, 0, 0, 1, 0, 0, 0, 0, 0, 1],  
       [1, 0, 0, 0, 0, 1, 1, 1, 0, 1],  
       [1, 0, 0, 0, 0, 0, 1, 1, 1, 0],  
       [0, 1, 1, 0, 0, 0, 1, 0, 0, 0]])
```


Explicit или implicit ?

```
'''
```

Пример 2. По 7 пользователям и 5 постам в соц.сети известна история реакций.

0 – посту поставлен дислайк, 1 – лайк, np.nan – пользователь не ставил реакцию у поста.

```
'''
```

```
num_nans = 10
```

```
matrix = np.random.randint(0, 2, (7, 5)).astype(float)
```

```
matrix.ravel()[np.random.choice(matrix.size, num_nans)] = np.nan
```

```
matrix
```

```
array([[ 0.,  1., nan,  0.,  0.],
       [ 0.,  0.,  1., nan,  1.],
       [ 1.,  0.,  0.,  1.,  0.],
       [ 1.,  1.,  1.,  0.,  1.],
       [ 0., nan,  1.,  1.,  0.],
       [ 1., nan,  0., nan,  0.],
       [nan, nan, nan, nan,  0.]])
```

Explicit или implicit ?

```
'''
```

Пример 3. По 3 пользователям и 8 ресторанам известна история отзывов на рестораны и их рейтинг по 5-балльной шкале. 1 – самая низкая оценка, ресторан очень не понравился, 5 – отличная оценка, np.nan – нет рейтинга и отзыва.

```
'''
```

```
matrix = np.random.randint(1, 6, (3, 8)).astype(float)
matrix.ravel()[np.random.choice(matrix.size, num_nans)] = np.nan
matrix
```

```
array([[ 1.,  3.,  3.,  4., nan,  5., nan, nan],
       [nan, nan, nan,  3.,  4.,  4.,  4.,  2.],
       [nan, nan,  2.,  1.,  4., nan,  5.,  4.]])
```

А здесь?

'''

Пример 4. По 5 клиентам есть градация по разным типам действий.

0.1 – просмотр

0.3 – просмотр + клик

0.5 – просмотр + клик + лайк

0.7 – просмотр + клик + комментарий

1 – просмотр + клик + лайк + комментарий

'''

В каком виде лучше хранить такую матрицу, когда у нас очень много нулей?

```
from scipy.sparse import coo_matrix, csr_matrix
```

```
import sys
```

```
matrix = random.randint(low=0, high=2, size=(10,10))
```

```
print(matrix)
```

```
print(f'Sparse матрица: {sys.getsizeof(csr_matrix(matrix))} байтов',  
      f'Обычный np.ndarray: {sys.getsizeof(matrix)} байтов',  
      sep=' \n' )
```

```
[[1 1 1 1 0 1 0 1 0 0]  
 [0 0 0 0 1 0 1 1 1 0]  
 [0 0 1 0 0 1 0 1 1 1]  
 [1 1 0 1 1 1 1 0 0 0]  
 [0 0 0 0 1 1 0 0 0 1]  
 [0 0 1 1 0 1 1 1 0 1]  
 [1 0 0 1 0 1 1 1 0 0]  
 [1 0 0 0 1 0 1 0 0 1]  
 [0 1 1 1 0 1 1 0 1 1]  
 [1 0 0 0 0 0 0 0 0 0]]
```

Sparse матрица: 64 байтов

Обычный np.ndarray: 920 байтов



Функции ранжирования

Функции ранжирования

1. Pointwise (точечные)

Например, решение через задачу регрессии или классификации.

$$\sum_{i=1}^l (a(x_i) - y(x_i))^2 \rightarrow \min$$

Функции ранжирования

1. Pointwise (точечные)

$$\sum_{x_i < x_j} [a(x_j) - a(x_i) < 0] \rightarrow \min$$

2. Pairwise (попарные)

Функции ранжирования

1. Pointwise (точечные)

$$\sum_{x_i < x_j} [a(x_j) - a(x_i) < 0] \rightarrow \min$$

2. Pairwise (попарные)

$$\sum_{x_i < x_j} L(a(x_j) - a(x_i)) \rightarrow \min$$

Функции ранжирования

1. Pointwise (точечные)

2. Pairwise (попарные)

3. Listwise (списочные)

RankNet:

$$\omega := \omega + \eta \frac{1}{1 + \exp(\langle \omega, x_j - x_i \rangle)} (x_j - x_i)$$

LambdaRank:

$$\omega := \omega + \eta \frac{1}{1 + \exp(\langle \omega, x_j - x_i \rangle)} (x_j - x_i) |\Delta nDCG_{ij}| (x_j - x_i)$$

Функции ранжирования

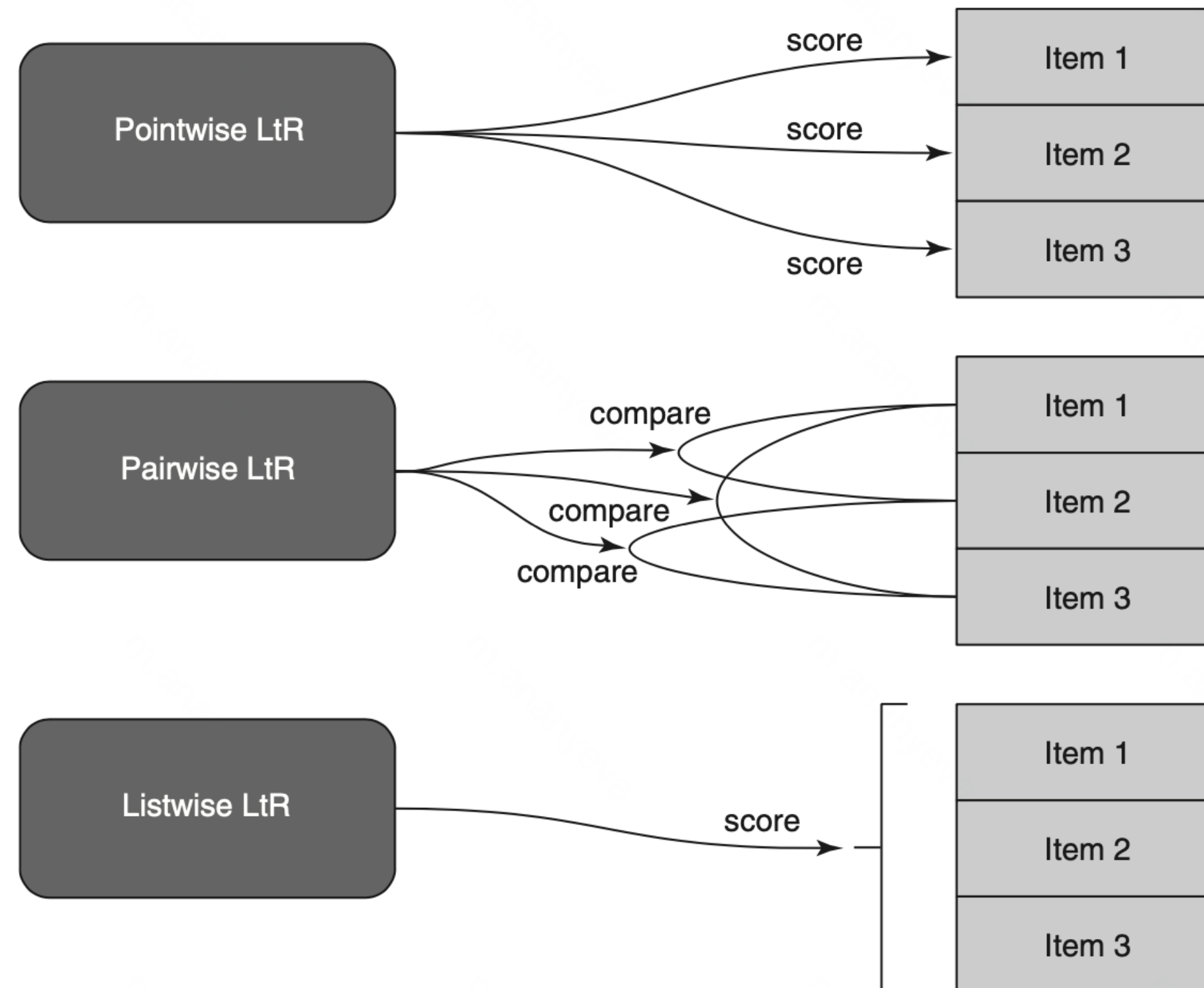


Figure 14.3 Different types of LTR algorithms

Примеры функций ранжирования

- **BPR** (Bayesian Personalised Ranking, pairwise)
- **WARP** (Weighted Approximate-Rank Pairwise)
- **RankNET** (pairwise)
- **LambdaRANK** (pairwise/listwise)
- **LambdaMART** (pairwise/listwise)
- **YetiRank**

BPR: Bayesian Personalized Ranking

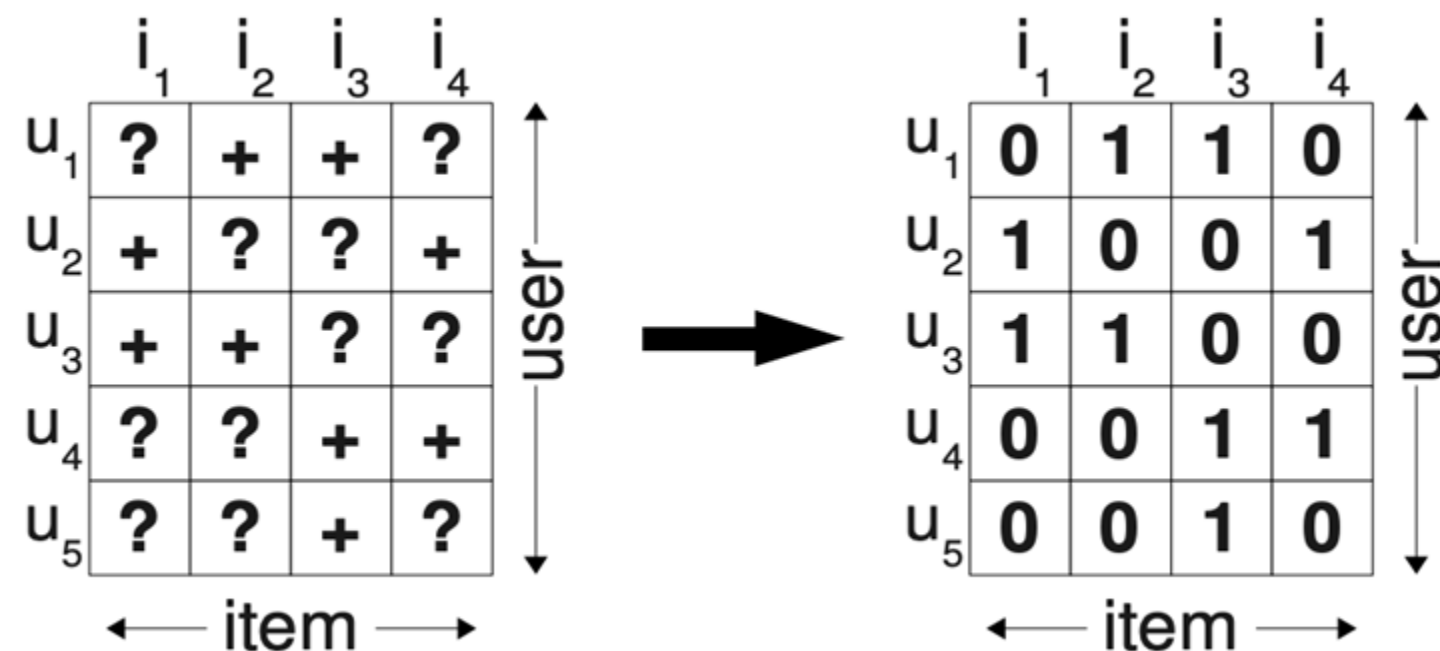


Рис. 1: Бинаризация implicit данных, где отсутствие интеракции кодируется как отрицательный пример.

BPR: Bayesian Personalized Ranking

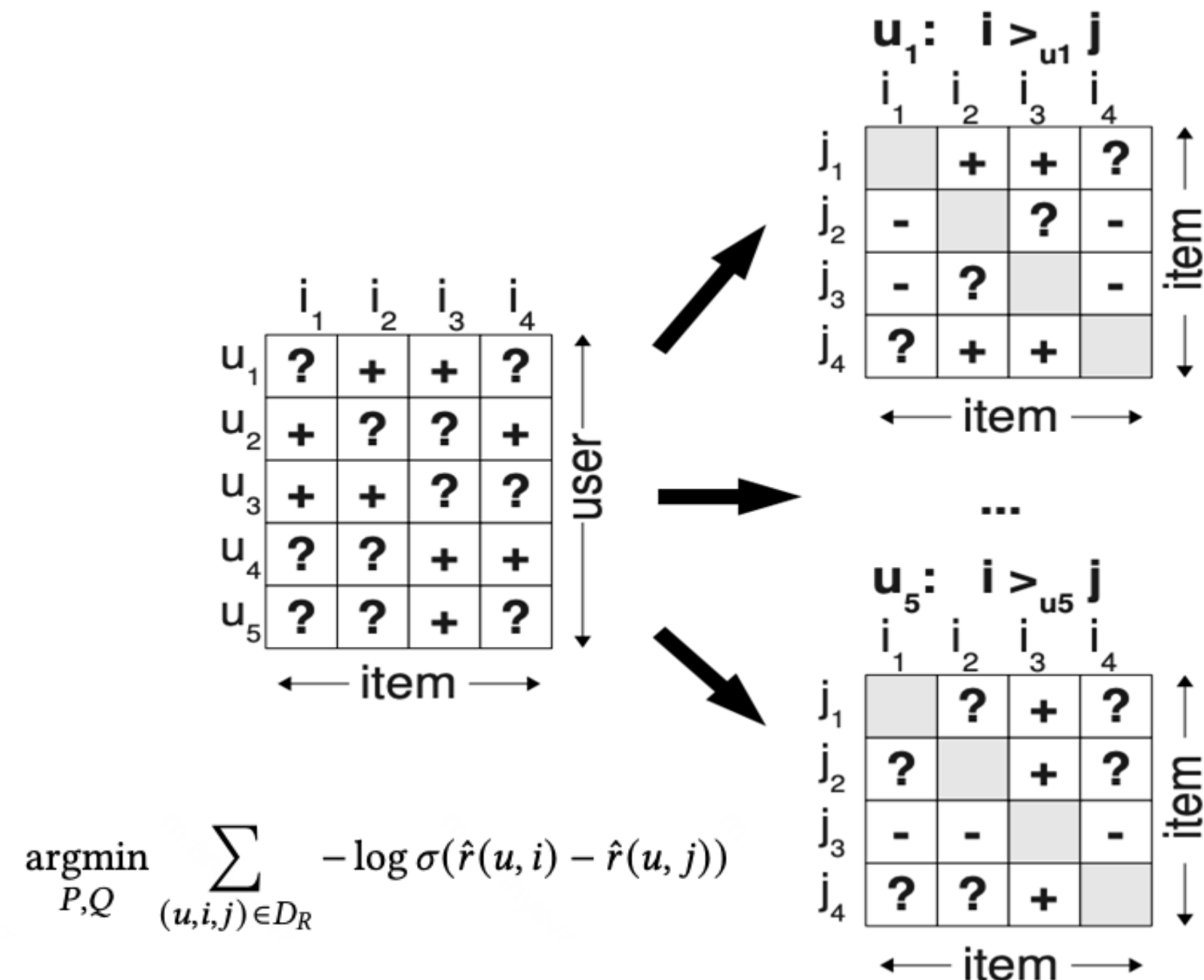
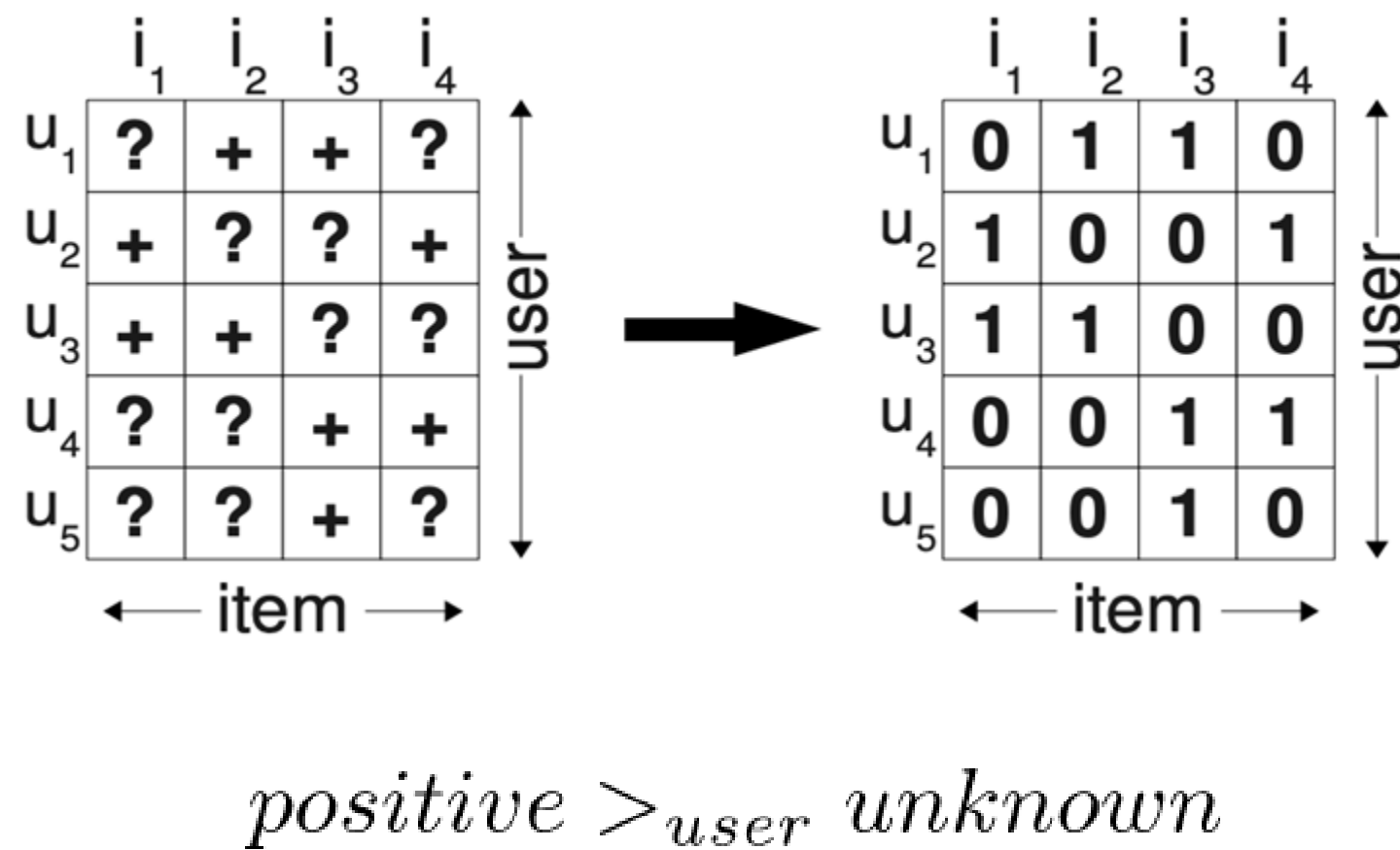


Рис. 1: Бинаризация implicit данных, где отсутствие интеракции кодируется как отрицательный пример.

Рис. 2: Парные предпочтения пользователю по объектам для BPR. + означает, что пользователь предпочитает i объект j -му, знак минуса - наоборот.

Revisiting BPR: A Replicability Study of a Common Recommender System Baseline

Aleksandr Milogradskii
alex.milogradsky@gmail.com
National Research University Higher
School of Economics, T-Bank
Russia

Oleg Lashinin
fotol764@gmail.com
Moscow Institute of Physics and
Technology, T-Bank
Russia

Alexander P
alexander.p.89@yandex.ru
Independent Researcher
Russia

Marina Ananyeva
ananyeva.me@gmail.com
National Research University Higher
School of Economics, T-Bank
Russia

Sergey Kolesnikov
scitator@gmail.com
T-Bank
Russia

ABSTRACT

Bayesian Personalized Ranking (BPR), a collaborative filtering approach based on matrix factorization, frequently serves as a benchmark for recommender systems research. However, numerous studies often overlook the nuances of BPR implementation, claiming that it performs worse than newly proposed methods across various tasks. In this paper, we thoroughly examine the features of the BPR model, indicating their impact on its performance, and investigate open-source BPR implementations. Our analysis reveals inconsistencies between these implementations and the original BPR paper, leading to a significant decrease in performance of up to 50% for specific implementations. Furthermore, through extensive experiments on real-world datasets under modern evaluation settings, we demonstrate that with proper tuning of its hyperparameters, the BPR model can achieve performance levels close to state-of-the-art methods on the top-n recommendation tasks and even outperform them on specific datasets. Specifically, on the Million Song Dataset, the BPR model with hyperparameters tuning statistically significantly outperforms Mult-VAE by 10% in NDCG@100 with binary relevance function.

ACM Reference Format:

Aleksandr Milogradskii, Oleg Lashinin, Alexander P, Marina Ananyeva, and Sergey Kolesnikov. 2024. Revisiting BPR: A Replicability Study of a Common Recommender System Baseline. In *18th ACM Conference on Recommender Systems (RecSys '24)*, October 14–18, 2024, Bari, Italy. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3640457.3688073>

1 INTRODUCTION

The issue of information overload, coupled with the rise of online services, has created a growing need for recommender systems [42]. These systems have attracted significant interest from both



Figure 1: Number of citations of BPR per year according to Google Scholar, as of May 15, 2024.

academic [1, 28, 61, 64] and industrial researchers [12, 27, 37], who have studied various methods associated with personalization.

Traditionally, recommender systems have relied on either explicit [7] or implicit [26] feedback. Explicit actions, such as user ratings, are relatively limited. In contrast, implicit feedback, such as views, clicks, and purchases, is vast in number, making it easier to build recommender systems based on these logs [49]. However, these models are less confident [26].

The advancements in simple linear models and matrix factorization techniques have played a crucial role in the development of recommender systems, a subfield of machine learning. In recent years, however, deep learning (DL) methods [35] have become dominant in other areas of machine learning, such as natural language processing (NLP) [10], computer vision (CV) [59], and reinforcement learning (RL) [31]. Given these advances, one might expect similar improvements in recommender systems. Contrarily, several studies on recommender systems suggest that linear models are more effective than DL models for top-n recommendation [15, 36, 53]. These linear methods are easier and faster to train than many deep

WARP:Weighted Approximate-Rank Pairwise*

Algorithm 1 K-OS algorithm for picking a positive item.

We are given a probability distribution P of drawing the i^{th} position in a list of size K . This defines the choice of loss function.

Pick a user u at random from the training set.

Pick $i = 1, \dots, K$ positive items $d_i \in \mathcal{D}_u$.

Compute $f_{d_i}(u)$ for each i .

Sort the scores by descending order, let $o(j)$ be the index into d that is in position j in the list.

Pick a position $k \in 1, \dots, K$ using the distribution P .

Perform a learning step using the positive item $d_{o(k)}$.

Algorithm 2 K-OS WARP loss

Initialize model parameters (mean 0, std. deviation $\frac{1}{\sqrt{m}}$).

repeat

 Pick a positive item d using Algorithm 1.

 Set $N = 0$.

repeat

 Pick a random item $\bar{d} \in \mathcal{D} \setminus \mathcal{D}_u$.

$N = N + 1$.

until $f_{\bar{d}}(u) > f_d(u) - 1$ or $N \geq |\mathcal{D} \setminus \mathcal{D}_u|$

if $f_{\bar{d}}(y) > f_d(u) - 1$ **then**

 Make a gradient step to minimize:

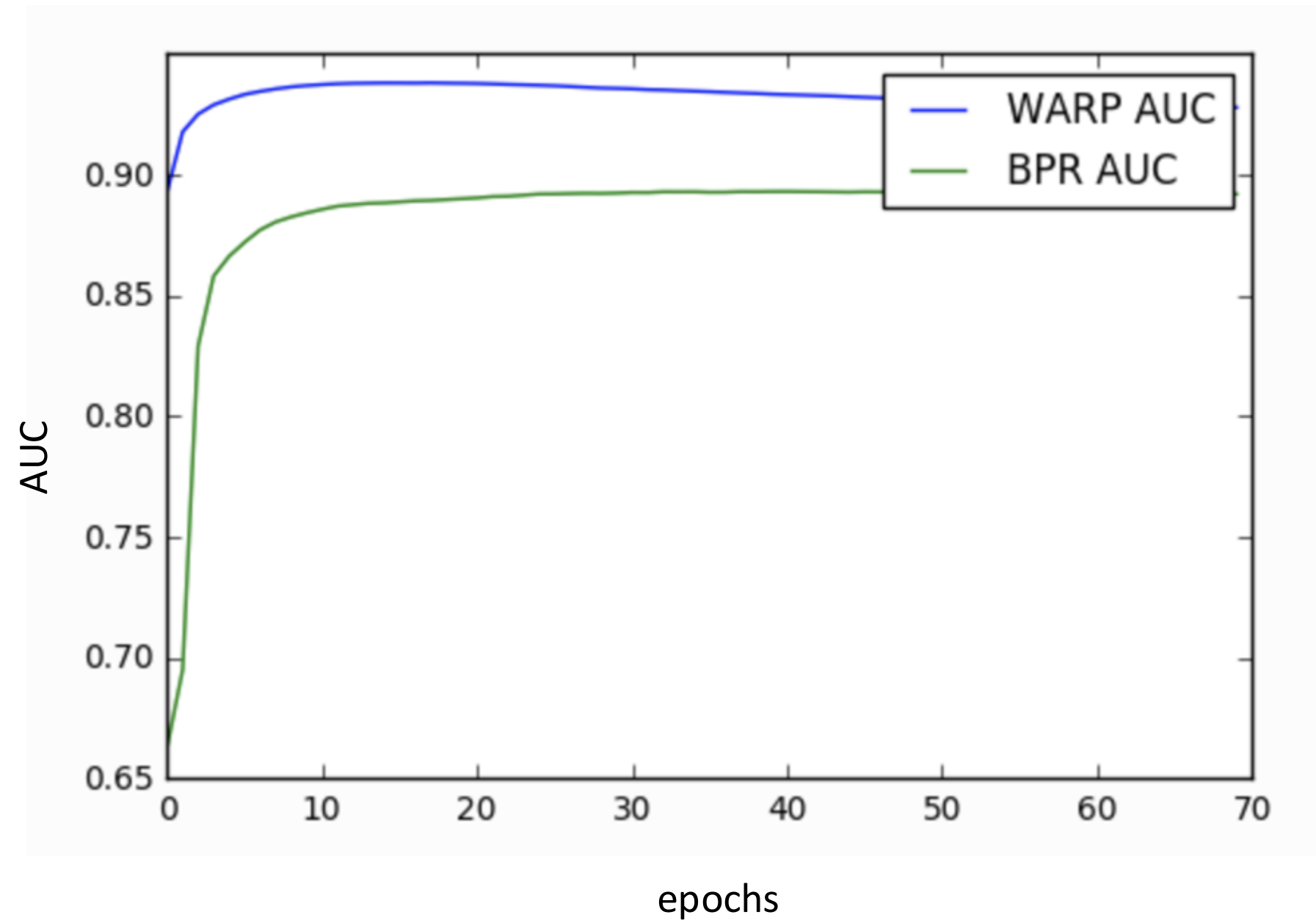
$$\Phi\left(\frac{|\mathcal{D} \setminus \mathcal{D}_u|}{N}\right) \max(0, 1 + f_{\bar{d}}(u) - f_d(u)).$$

 Project weights to enforce constraints, e.g. if $\|V_i\| > C$ then set $V_i \leftarrow (CV_i)/\|V_i\|$.

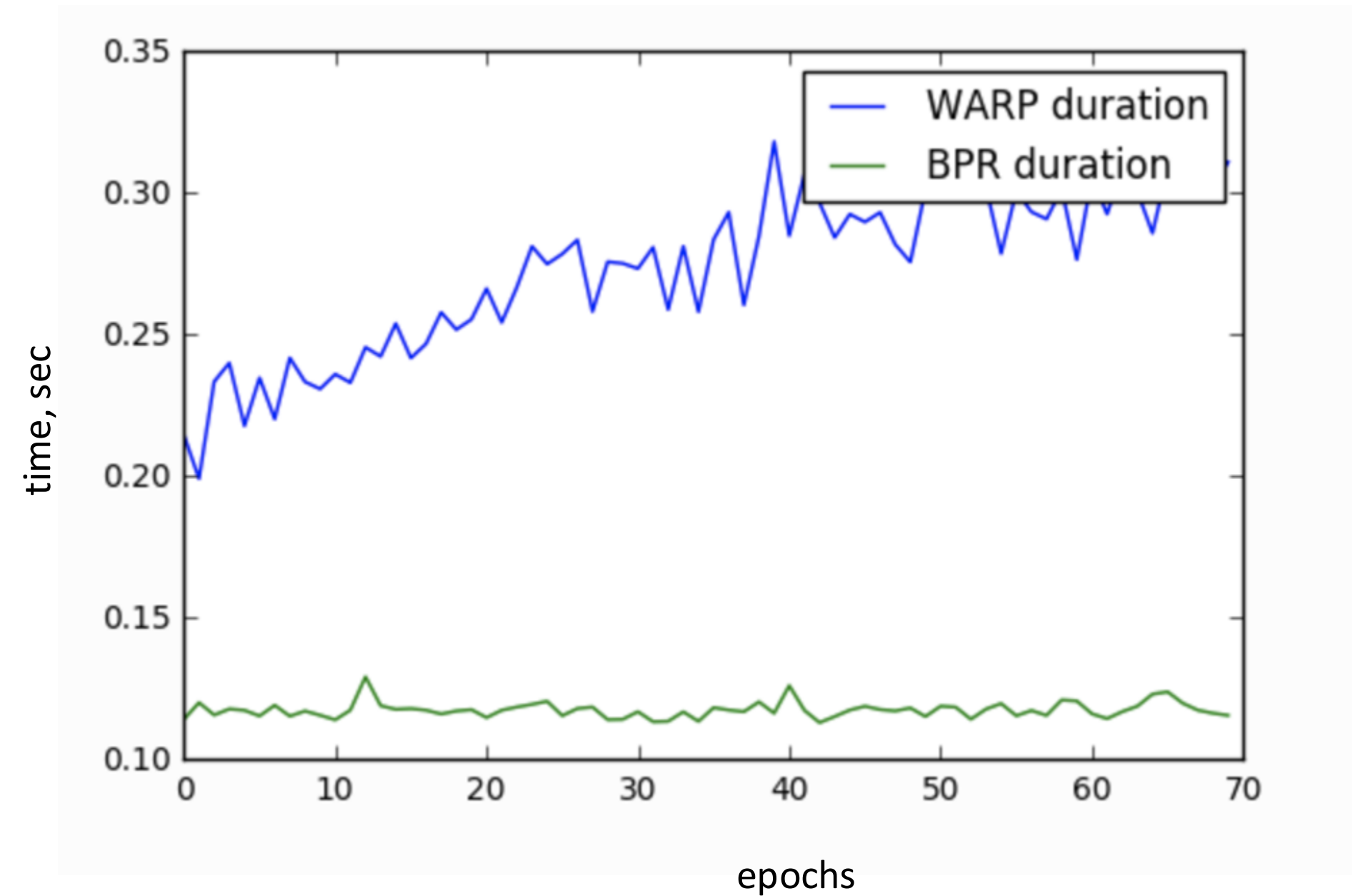
end if

until validation error does not improve.

WARP vs. BPR*



Обычно, WARP лучше по качеству...



но более медленный =(



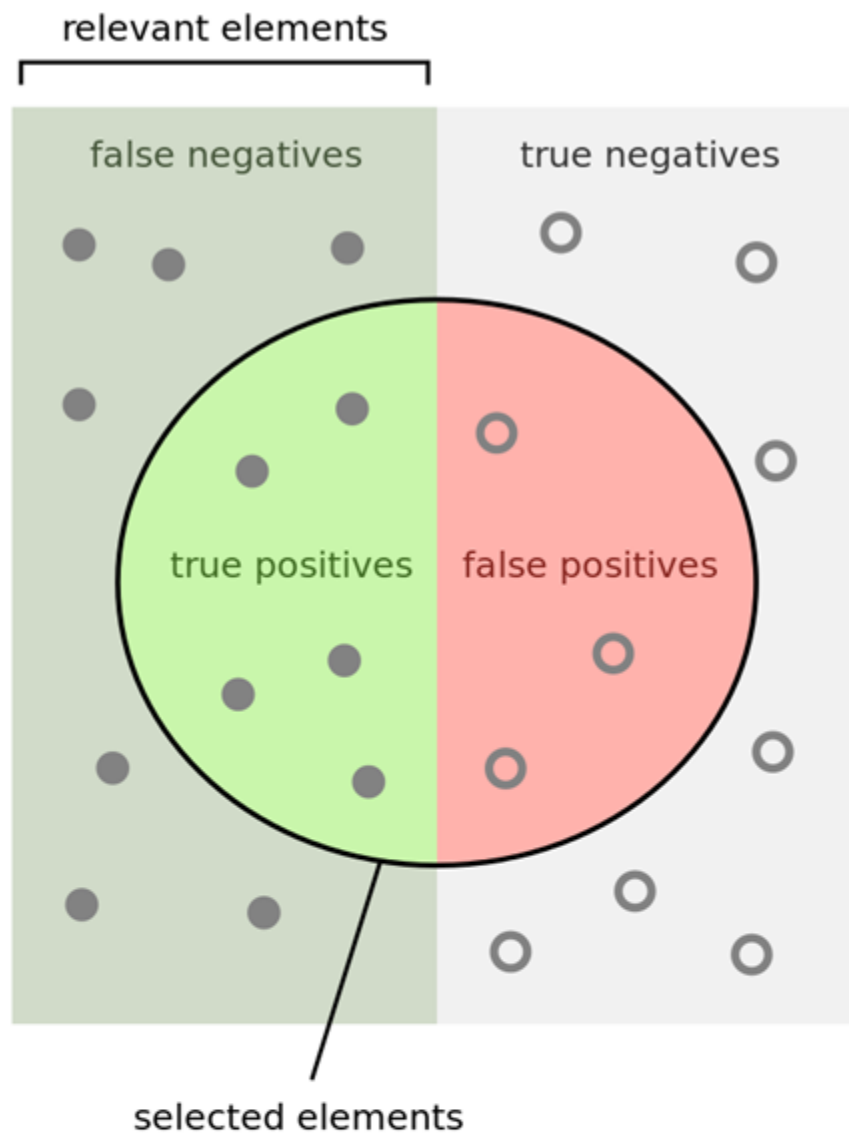
Метрики оценки качества ранжирования

Зоопарк метрик качества ранжирования

- **Hitrate@k (Hit ratio)**
- **Precision@k**
- **Recall@k**
- **AP@k, MAP@k, MNAP@k**
- **DCG@k, NDCG@k**
- **MRR**

и другие :)

Hitrate, Precision@k, Recall@k



$$\text{Hit Rate} = \frac{\# \text{ hits}}{\# \text{ hits} + \# \text{ misses}}$$

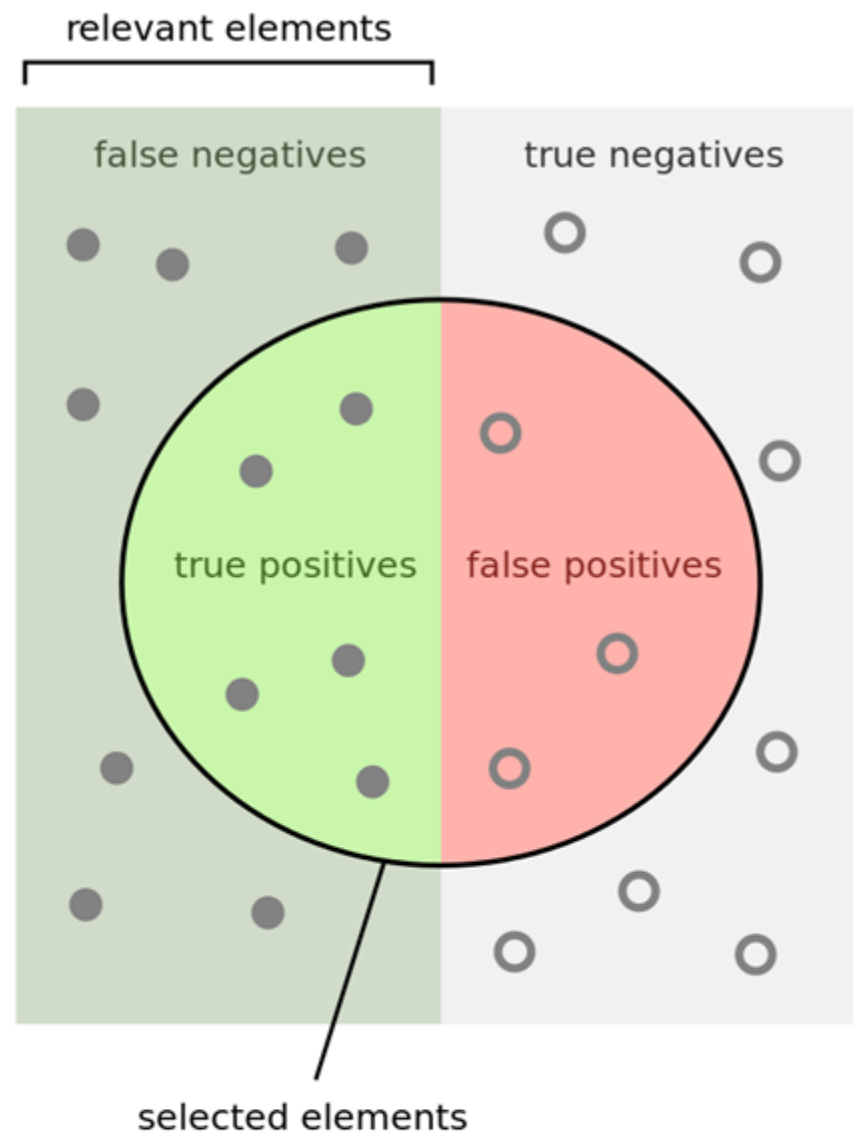
How many selected items are relevant?

Precision = $\frac{\text{green semi-circle}}{\text{green semi-circle} + \text{red semi-circle}}$

How many relevant items are selected?

Recall = $\frac{\text{green semi-circle}}{\text{green semi-circle} + \text{dark grey rectangle}}$

Hitrate, Precision@k, Recall@k



$$\text{Hit Rate} = \frac{\# \text{ hits}}{\# \text{ hits} + \# \text{ misses}}$$

Precision – ?

Recall – ?

Эти метрики зависят от количества рекомендаций для каждого пользователя?

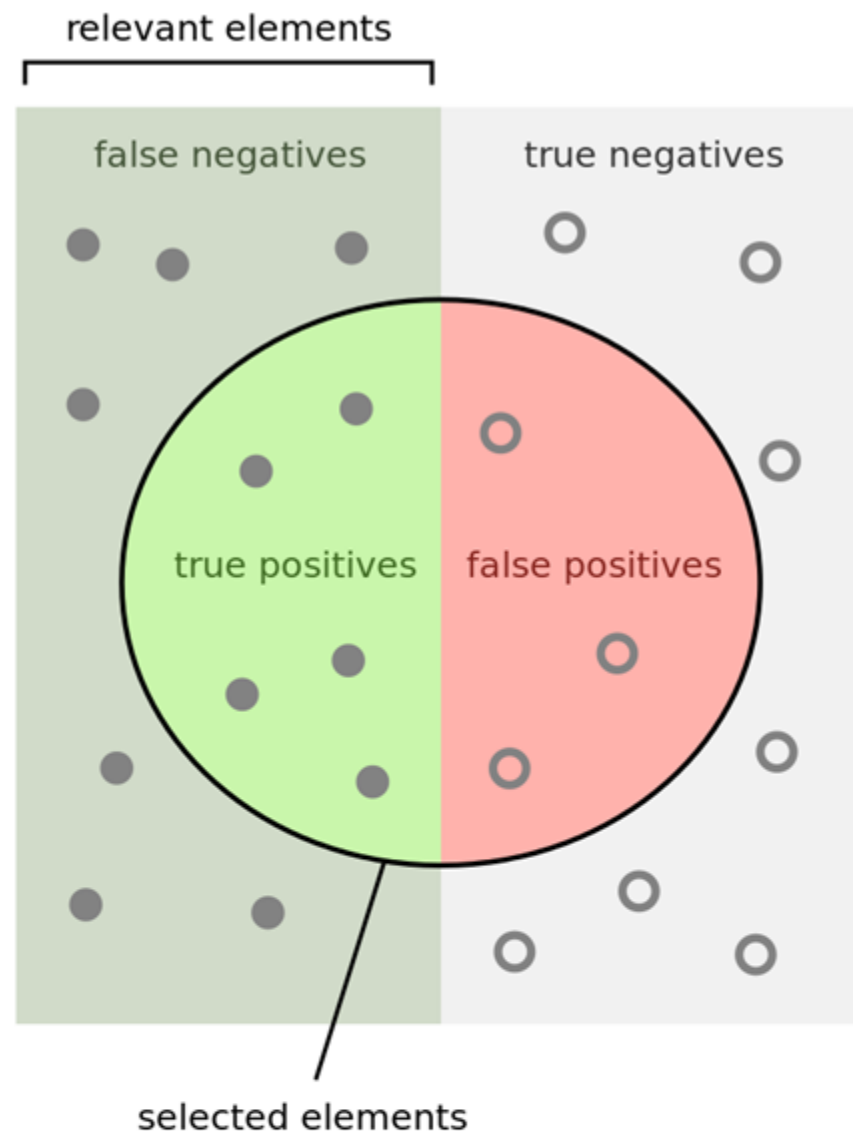
How many selected items are relevant?

Precision =

How many relevant items are selected?

Recall =

Hitrate, Precision@k, Recall@k



How many selected items are relevant?

Precision = $\frac{\text{Green semi-circle}}{\text{Green and Red semi-circles}}$

How many relevant items are selected?

Recall = $\frac{\text{Number of relevant items selected}}{\text{Total number of relevant items in the dataset}}$

$$Hit\ Rate = \frac{\# hits}{\# hits + \# misses}$$

Precision – сколько из рекомендованных объектов оказались релевантными.

Recall – сколько из релевантных для пользователя объектов мы порекомендовали.

Precision@k, Recall@k – фиксируем для каждого пользователя топ k рекомендованных объектов, считаем метрику по бинарному таргету (0 – нерелевантная рекомендация, 1 – релевантная).

Hitrate@k

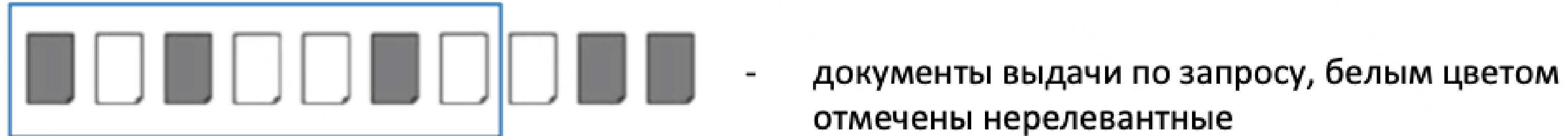
$$\text{Hit Rate} = \frac{\# \text{ hits}}{\# \text{ hits} + \# \text{ misses}}$$

1.1. $\text{Hitrate}@k = \frac{\sum_{i=1}^k [y_i = 1]}{|Y_k|}$ - для 1 юзера
~ accuracy (доля правильных ответов)

1.2. На юзере $\text{hr}@k=1$, если хотя бы 1 правильный, k - длина списка рек-и
 $\text{Hitrate}@k = \frac{1}{|U|} \sum_{u \in U} [\sum_{i=1}^k [y_{ui} = 1] > 0]$ - доля юзеров, у кого хотя бы 1 рек-и правильная

	1	2	3	4	5	6	7	
u_1								 - нерелевантный
u_2								 - релевантный
u_3								$ I = 7$

Посчитаем precision@k , recall@k



Посчитаем precision@k, recall@k



- документы выдачи по запросу, белым цветом отмечены нерелевантные

y_i - истинная релевантность, $y \in Y$
айтенов на i -й позиции

$\hat{y}_i$ - предсказанная релевантность

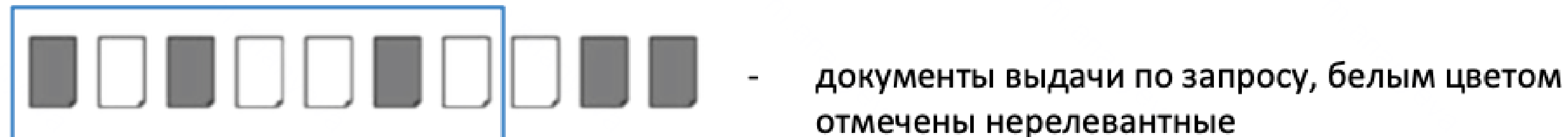
$$\text{precision@k} = \frac{1}{k} \sum_{i=1}^k [y_i = 1],$$

где k - длина списка рекомендаций

$$\text{recall@k} = \frac{\sum_{i=1}^k [y_i = 1]}{\sum_{i=1}^N [y_i = 1]} = \frac{\sum_{i=1}^k [\hat{y}_i = 1]}{|Y|},$$

где N - мн-во всех айтенов

Посчитаем precision@k, recall@k

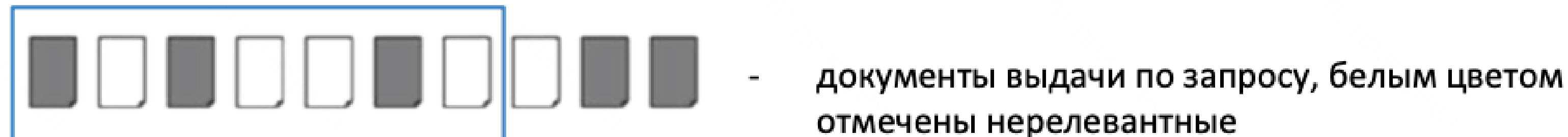


Пусть $k=7$

$$precision@7 = \frac{\# \text{ predicted relevant}}{k \text{ predicted}}$$

$$recall@7 = \frac{\# \text{ predicted relevant}}{\# \text{ relevant}}$$

Посчитаем precision@k, recall@k

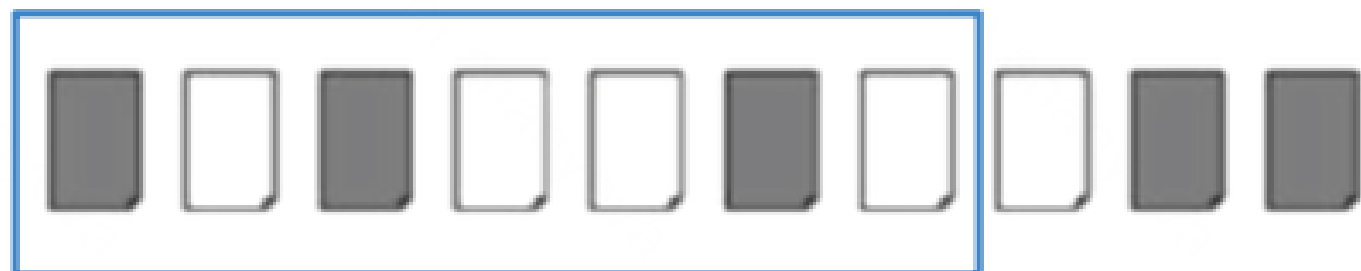


Пусть $k=7$

$$\text{precision@7} = \frac{\# \text{ predicted relevant}}{k \text{ predicted}} = \frac{3}{7}$$

$$\text{recall@7} = \frac{\# \text{ predicted relevant}}{\# \text{ relevant}} = \frac{3}{5}$$

Посчитаем precision@k, recall@k



Плюсы:

- + интерпретируемость
- + простота
- + применимо и к бинар. классификации
- + расчет и на юзере, и на всей выборке

Минусы:

- чувствительность к порогу k
- не учитывают порядок
- бинаризуем оценки релевантности

Пример:

1	2	3	4	5
■	■	□	□	□

 ■ - релевантный
□ - нерелевантный

$$p@5 = \frac{2}{5}$$

1	2	3	4	5
□	□	□	■	■

$$p@5 = \frac{2}{5}$$

Average precision@ k (AP@k)

- ⑩ Учитывает порядок рекомендованных объектов в топе k.

$$AP@k = \sum_{i=1}^k \frac{y_i}{\underbrace{\sum_{j=1}^k y_j}_{\text{кол-во релевантных айтемов}}} \cdot p@i = \frac{\sum_{i=1}^k y_i \cdot p@i}{\sum_{i=1}^k y_i}$$

Average precision@ k (AP@k)

- ⑩ Учитывает порядок рекомендованных объектов в топе k.

$$AP@k = \sum_{i=1}^k \frac{y_i}{\sum_{j=1}^k y_j} \cdot p@i = \frac{\sum_{i=1}^k y_i \cdot p@i}{\sum_{i=1}^k y_i}$$

*кон-во релевантных
итемов*

Пример:

1	2	3	4	5
■	■	□	□	□

 ■ - релевантный

Average precision @ k (AP@k)

- ⑩ Учитывает порядок рекомендованных объектов в топе k.

$$AP@k = \sum_{i=1}^k \frac{y_i}{\underbrace{\sum_{j=1}^k y_j}_{\text{кол-во релевантных объектов}}} \cdot p@i = \frac{\sum_{i=1}^k y_i \cdot p@i}{\sum_{i=1}^k y_i}$$

Пример:

	1	2	3	4	5	
						 - релевантный
p@	1	1	$\frac{2}{3}$	$\frac{2}{4}$	$\frac{2}{5}$	 - нерелевантный
y _i	1	1	0	0	0	total=2

$$AP@5 = \frac{1+1+0+0+0}{2} = 1$$

Average precision @k (AP@k)

- ⑩ Учитывает порядок рекомендованных объектов в топе k.

$$AP@k = \sum_{i=1}^k \frac{y_i}{\sum_{j=1}^k y_j} \cdot p@i = \frac{\sum_{i=1}^k y_i \cdot p@i}{\sum_{i=1}^k y_i}$$

$\underbrace{\sum_{j=1}^k y_j}_{\text{коп-во релевантных объектов}}$

Пример:

	1	2	3	4	5	
						- релевантный
p@	1	1	$\frac{2}{3}$	$\frac{2}{4}$	$\frac{2}{5}$	- нерелевантный
y _i	1	1	0	0	0	total=2

$$AP@5 = \frac{1+1+0+0+0}{2} = 1$$

	1	2	3	4	5
p@	0	0	0	$\frac{1}{4}$	$\frac{2}{5}$
y _i	0	0	0	1	1

$$AP@5 = \frac{0+0+0+\frac{1}{4}+\frac{2}{5}}{2} = \frac{13}{40} = 0.325$$

Mean average precision (MAP@k)

$$\text{MAP@}k = \frac{1}{|U|} \sum_{u \in U} \text{AP@}k_u,$$

где U - множество пользователей

Зачем нужен $\text{MNAP@}k$?

$$\text{MNAP@}k = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\min(n_u, k)} \cdot \sum_{i=1}^k y_i \cdot p@i_u$$

где n_u - кол-во айтелев, с которыми пользователь взаимодействовал

Плюс : + учитывает порядок

Минус : $|Y_u| \leq k \Rightarrow$ метрика занижается при большой k

Mean average precision (MAP@k)

$$\text{MAP@}k = \frac{1}{|U|} \sum_{u \in U} \text{AP@}k_u,$$

где U - множество пользователей

Зачем нужен $\text{MNAP@}k$?

$$\text{MNAP@}k = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\min(n_u, k)} \cdot \sum_{i=1}^k y_i \cdot p@i_u$$

где n_u - кол-во айтелев, с которыми пользователь взаимодействовал

Плюс : + учитывает порядок

Минус : $|Y_u| \leq k \Rightarrow$ метрика занижается при большой k

Mean average precision (MAP@k)

	1	2	3	4	5	6	7	
u_1								$ I =7$
u_2								
u_3								

$$\text{MAP@3} = \frac{1}{3} \cdot \left(\frac{1}{2} + \frac{5}{6} + \frac{7}{12} \right) = \frac{23}{36} \approx 0.64$$

$$\text{AP@3}_{u_2} = \frac{1 + \frac{1}{2} \cdot 0 + \frac{2}{3} \cdot 1}{2} = \frac{5}{6}$$

$$\text{MNAP@3} = \frac{1}{3} \cdot \left(\frac{1}{2} \cdot \frac{1}{2} + \frac{1}{2} \cdot \frac{5}{3} + \frac{1}{2} \cdot \frac{7}{6} \right) = \frac{5}{9} \approx 0.56$$

Discounted Cumulative Gain (DCG@k)

Чем выше релевантные элементы в списке рекомендаций, тем лучше.

$$DCG@k = \sum_{i=1}^k g(y_i) d(i) = \sum_{i=1}^k \frac{2^{y_i} - 1}{\log(i+1)}$$

$g(y_i)$ - функция прироста, gain
 $d(i)$ - регуляризация (штраф), discount

экспоненциальный прирост при линейном росте релевантности
чем выше релевантный элемент в выдаче, тем больше штраф

$$NDCG@k = \frac{DCG@k}{IDCG@k}, \text{ где } IDCG@k - \text{идеальное (max) значение } DCG@k$$
$$IDCG@k = \sum_{i=1}^k \frac{1}{\log(i+1)}$$

NDCG@k

Пример вычисления DCG и nDCG:

Дано множество документов, где каждый документ оценивается от 3 до 0, где 3 — очень релевантен, а 0 — не релевантен. Пусть таким множеством будет $S = \{D_1, D_2, D_3, D_4, D_5, D_6\}$, где оценка релевантности по опросу пользователей задается (в том же порядке) множеством $R = \{3, 2, 3, 0, 1, 2\}$.

Тогда $DCG@6 = \sum_{i=1}^6 \frac{rel_i}{\log(i+1)} = 3 + 1.262 + 1.5 + 0 + 0.387 + 0.712 = 6.861$.

i	rel_i	$\log(i + 1)$	$\frac{rel_i}{\log(i+1)}$
1	3	1	3
2	2	1.585	1.262
3	3	2	1.5
4	0	2.322	0
5	1	2.585	0.387
6	2	2.807	0.712

NDCG@k

Идеальный порядок оценок релевантности $Ideal = \{3, 3, 2, 2, 1, 0\}$. DCG для данного множества будет следующим:

$$maxDCG@6 = \sum_{i=1}^6 \frac{rel_i}{\log(i+1)} = 3 + 1.893 + 1 + 0.861 + 0.387 + 0 = 7.141.$$

i	rel_i	$\log(i + 1)$	$\frac{rel_i}{\log(i+1)}$
1	3	1	3
2	3	1.585	1.893
3	2	2	1
4	2	2.322	0.861
5	1	2.585	0.387
6	0	2.807	0

$$\text{Итого } nDCG@6 = \frac{DCG@6}{maxDCG@6} = \frac{6.861}{7.141} = 0.961.$$

Дополнительные метрики измерения качества

Diversity (разнообразие) - число рекомендаций из разных категорий, степень различия рекомендаций между сессиями пользователя, различие рекомендаций между пользователями.

Novelty (новизна) - сколько среди рекомендаций новых для пользователей объектов.

Coverage (покрытие) - доля объектов, которые хотя бы раз побывали среди топа рекомендаций.

Serendipity (догадливость) - способность угадывать неожиданные нетривиальные предпочтения пользователя.

Рекомендованные источники

Книги:

1. [Recommender Systems Handbook](#). Francesco Ricci et al, Springer, 2022
2. [Hands-On Recommendation Systems with Python](#). Rounak Banik, 2018
3. [Building Recommendations Systems in Python and JAX](#). Brian Bischof et al, 2018
4. [Practical Recommender Systems](#). Kim Falk, 2019
5. [Session-Based Recommender Systems Using Deep Learning](#). Reza Ravanmehr et al. Springer, 2024
6. [Designing Machine Learning Systems](#). Chip Huyen, 2022
7. [Designing Data-Intensive Applications](#). Martin Kleppmann, 2017

Рекомендованные источники

Конференции:

1. **ACM Recommender Systems**
2. **UMAP (Conference on User Modeling, Adaptation and Personalization)**
3. **ECIR (European Conference on Information Retrieval)**
4. **SIGIR (Conference on Research and Development in Information Retrieval)**
5. **WSDM (ACM International Conference on Web Search and Data Mining)**
6. **WWW (The Web Conference)**

Полный список можно посмотреть [тут](#).

Рекомендованные источники

Интересные каналы про RecSys для профессионального сообщества:

1. https://t.me/ods_recommender_systems
2. https://t.me/recsys_for_all
3. https://t.me/knowledge_accumulator
4. <https://t.me/inforetriever>
5. <https://t.me/ml4value>
6. <https://t.me/researchoshnaya>
7. <https://t.me/WazowskiRecommends>
8. <https://t.me/wildrecsys>
9. <https://t.me/mlvok>
10. <https://t.me/RecSysChannel>